

Bloque 7

Software robótico: ROS 2, simulación y planificación de movimiento

82514, Mecatrónica y Robótica · IQS Universitat Ramon Llull

Apuntes del profesor con citas de fuente · Sesiones S32–S37 (25, 26 y 27 de noviembre; 2, 3 y 4 de diciembre de 2026) · 8 horas

Ediciones citadas: Corke (2023), Robotics, Vision and Control, 3.ª ed. Python, Springer · Lynch y Park (2017), Modern Robotics, Cambridge · Thrun, Burgard y Fox (2005), Probabilistic Robotics, MIT Press · documentación oficial de ROS 2 (Jazzy Jalisco), Gazebo, Nav2 y MoveIt 2

Objetivos de aprendizaje y convenciones

- Explicar la arquitectura de ROS 2, nodos, topics, servicios, acciones, parámetros y el grafo de computación sobre DDS, y razonar la elección de políticas QoS para un canal de datos concreto.
- Crear un workspace, construir un paquete Python con colcon y escribir un nodo publicador/suscriptor con rclpy; simular un robot en Gazebo y leer e interpretar el árbol de transformadas TF2.
- Formular el problema de planificación de movimiento en el espacio de configuraciones y comparar, con criterios de completitud, optimalidad y coste computacional, los planificadores basados en rejilla (transformada de distancia, A^* , D^*), los de muestreo (PRM, RRT) y los campos potenciales.
- Describir la arquitectura de Nav2 (servidores de planificación y control, costmaps por capas, behavior trees) y de MoveIt 2 (move_group, planning scene, OMPL), y conectarlas con los algoritmos y con la localización AMCL del bloque 6.

Cómo leer las citas

Para los algoritmos de planificación se usa el formato (Autor, año, p. X) con página verificada sobre las ediciones de la carpeta de bibliografía: Corke (2023), cap. 5, pp. 161-214; Lynch y Park (2017), cap. 10, pp. 353-398; y Thrun, Burgard y Fox (2005), cap. 9, pp. 281-307. Para ROS 2, Gazebo, Nav2 y MoveIt 2 no existe edición paginada estable: se citan por el nombre del documento y la sección de la documentación oficial, docs.ros.org (versión LTS Jazzy Jalisco), gazebosim.org/docs, docs.nav2.org y moveit.picknik.ai, sin número de página, y así debe entenderse cada mención «(docs ...)». Las comillas «» marcan traducción casi literal. El guion de cada clase lista los puntos a tratar explícitamente en cada segmento; no es una transcripción.

Sesión S32 (mié 25 nov, 1 h), Arquitectura de ROS 2: nodos, topics, DDS y QoS

Guion de la clase

0-5 min Motivación: por qué un middleware

- Recuperar el hilo del curso: en B4 modelamos poses y cinemática, en B5 control, en B6 percepción y localización; hoy empieza el software que integra todo eso en un robot real.
- Plantear el problema de ingeniería: decenas de procesos (drivers, filtros, planificadores, control) escritos por equipos distintos que deben intercambiar datos con latencias y fiabilidades diferentes, la solución del ecosistema es ROS 2, un «conjunto de bibliotecas software y herramientas para construir aplicaciones robóticas» (docs.ros.org, portada de ROS 2 Jazzy).
- Aclarar qué NO es ROS 2: no es un sistema operativo ni un lenguaje; es middleware de comunicación + herramientas + convenciones + ecosistema de paquetes.

5-20 min Nodos y el grafo de computación

- Definir nodo: un participante del grafo que «debe ser responsable de un único propósito modular» (docs.ros.org, tutorial Understanding nodes); un ejecutable puede contener varios nodos.
- Dibujar en la pizarra el grafo de computación de un AMR mínimo: nodo driver del LiDAR → topic /scan → nodo de localización → topic /amcl_pose; nodo planificador → topic /cmd_vel → nodo driver de motores; ir anotando qué es nodo (círculo) y qué es topic (rectángulo), este dibujo se reutiliza toda la semana.
- Enunciar la idea arquitectónica: el grafo de computación es el conjunto de nodos y sus canales de comunicación; los nodos se descubren entre sí automáticamente, sin maestro central, diferencia clave con ROS 1, donde existía roscore (docs.ros.org, concepto The ROS 2 graph).

20-40 min Topics y publicación/suscripción

- Definir topic como bus con nombre sobre el que los nodos intercambian mensajes tipados; el patrón es publicación/suscripción anónima y de muchos a muchos: el publicador no sabe quién escucha (docs.ros.org, tutorial Understanding topics).
- Mensajes: estructuras tipadas definidas en ficheros .msg; ejemplos que verán en el taller: sensor_msgs/msg/LaserScan, geometry_msgs/msg/Twist, nav_msgs/msg/Odometry.
- Demo en vivo (5 min, con el portátil del aula): ros2 run demo_nodes_cpp talker en una terminal y ros2 run demo_nodes_py listener en otra; enseñar que son procesos independientes y hasta de lenguajes distintos.
- Demo de introspección: ros2 node list, ros2 topic list -t, ros2 topic echo /chatter, ros2 topic hz /chatter y rqt_graph para ver el grafo dibujado, comparar con el dibujo de la pizarra.
- Subrayar el valor mecatrónico del desacoplo: puedo sustituir el nodo simulado por el driver real sin tocar el resto del grafo si respeto nombres y tipos de topics.

40-55 min DDS y calidad de servicio (QoS)

- Explicar que ROS 2 se apoya en DDS (Data Distribution Service), estándar industrial de comunicación en tiempo real, a través de una capa de abstracción (rmw) con varios proveedores intercambiables (docs.ros.org, concepto About different ROS 2 DDS/RMW vendors).
- Presentar las políticas QoS principales con su caso de uso: reliability (best effort para el LiDAR a 40 Hz, donde un barrido perdido no importa; reliable para comandos), durability (transient local para el mapa, que un suscriptor tardío debe recibir), history/depth y deadline (docs.ros.org, concepto About Quality of Service settings).

- Regla práctica que enunciar tal cual: publicador y suscriptor solo conectan si sus QoS son compatibles, el error silencioso más frecuente en proyectos ROS 2 («los topics están, pero no llega nada»).
- Ejercicio rápido a mano alzada: elegir QoS para /scan, /cmd_vel y /map y justificarlo.

55-60 min Cierre

- Anunciar S33 (servicios, acciones, launch, colcon) y encargar el deber previo al taller de S34: traer el portátil con ROS 2 Jazzy instalado (nativo Ubuntu 24.04 o contenedor docker proporcionado) y verificado con el talker/listener de hoy.

32.1. De programa único a grafo de computación

Un robot de complejidad realista no es un programa: es un sistema distribuido. Sobre la misma máquina, o sobre varias, conviven drivers de sensores que producen datos a decenas de hercios, filtros de estimación, planificadores que piensan a segundos y lazos de control que actúan a milisegundos, escritos por personas distintas, en momentos distintos y a menudo en lenguajes distintos. El middleware robótico existe para que ese conjunto se comporte como un sistema: da un modelo de comunicación común, herramientas de introspección y convenciones de empaquetado. ROS 2 (Robot Operating System 2) es hoy el estándar de facto: pese a su nombre no es un sistema operativo, sino un conjunto de bibliotecas y herramientas, comunicación, drivers, algoritmos, visualización, build system, para construir aplicaciones robóticas (docs.ros.org, portada y sección de conceptos de ROS 2 Jazzy). En este curso usamos la versión LTS Jazzy Jalisco sobre Ubuntu 24.04, la combinación soportada en el laboratorio.

La unidad de organización es el nodo. Según la documentación oficial, «cada nodo en ROS debe ser responsable de un único propósito modular, por ejemplo, controlar los motores de las ruedas o publicar los datos de un telémetro láser» (docs.ros.org, tutorial Understanding nodes). Los nodos se comunican entre sí mediante topics, servicios, acciones y parámetros, y el conjunto de nodos y conexiones activas en un instante forma el grafo de computación. A diferencia de ROS 1, en ROS 2 no existe un proceso maestro central: los nodos se descubren entre sí mediante el mecanismo de descubrimiento distribuido de DDS, lo que elimina el punto único de fallo y permite arrancar y detener partes del sistema en cualquier orden. Para el aula conviene insistir en la consecuencia de diseño: la arquitectura de un sistema ROS 2 se decide eligiendo qué nodos existen y qué interfaces (nombres y tipos de mensaje) los conectan, exactamente igual que en el bloque 1 decidíamos la arquitectura mecatrónica repartiendo funciones entre subsistemas.

32.2. Topics, mensajes y el patrón publicación/suscripción

Los topics son el mecanismo de comunicación principal: buses con nombre (por convención jerárquico, como /scan o /cmd_vel) por los que fluyen mensajes de un tipo fijo. El patrón es publicación/suscripción: un nodo publica sin saber quién escucha y cualquier número de nodos puede suscribirse sin que el publicador lo sepa; la relación es de muchos a muchos y anónima (docs.ros.org, tutorial Understanding topics). Los mensajes son estructuras de datos tipadas definidas en ficheros de interfaz .msg e independientes del lenguaje: el mismo sensor_msgs/msg/LaserScan lo produce un driver en C++ y lo consume un nodo Python. Este desacoplo es la propiedad de ingeniería más valiosa del modelo: el nodo de localización del bloque 6 no distingue si /scan procede del LiDAR real o de Gazebo, y esa sustituibilidad, simulación y realidad intercambiables tras la misma interfaz, es la base metodológica del taller de S34 y del proyecto integrador.

La contrapartida del desacoplo es que las interfaces se convierten en el contrato del sistema, y por eso ROS 2 dedica tanto esfuerzo a la introspección: `ros2 node list` y `ros2 node info` enumeran nodos y sus conexiones; `ros2 topic list -t`, `ros2 topic echo` y `ros2 topic hz` muestran qué se publica, con qué

contenido y a qué frecuencia; `ros2 interface show` imprime la definición de un tipo de mensaje; y `rqt_graph` dibuja el grafo de computación completo (docs.ros.org, tutoriales de la serie Beginner: CLI tools). Un ingeniero que domina estas herramientas puede diagnosticar un robot ajeno sin leer una línea de su código; en el laboratorio serán la primera herramienta de depuración.

32.3. DDS y las políticas de calidad de servicio

Bajo la API de ROS 2 no hay un protocolo casero, sino DDS (Data Distribution Service), un estándar OMG de comunicación publish/subscribe en tiempo real usado en aviónica, ferroviario y defensa. ROS 2 accede a DDS a través de una capa de abstracción del middleware (rmw), de modo que pueden intercambiarse implementaciones de distintos proveedores, Fast DDS es la implementación por defecto en Jazzy, con Cyclone DDS y otras como alternativas, sin cambiar el código de usuario (docs.ros.org, concepto About different ROS 2 DDS/RMW vendors). La herencia más importante de DDS para el usuario son las políticas de calidad de servicio (QoS), que convierten decisiones antes implícitas en parámetros explícitos de cada publicador y suscriptor (docs.ros.org, concepto About Quality of Service settings): reliability (reliable garantiza la entrega con retransmisiones; best effort no), durability (transient local conserva los últimos mensajes para suscriptores que llegan tarde; volatile no), history y depth (cuántos mensajes retener en cola) y deadline (periodo máximo esperado entre mensajes).

La elección de QoS es una decisión de ingeniería con la misma estructura que el dimensionado de un bus de campo en el bloque 3: un flujo de sensor de alta frecuencia como `/scan` se publica best effort, si se pierde un barrido, el siguiente llega en 25 ms y retransmitir solo añadiría latencia, mientras que un mapa que se publica una vez debe ser reliable y transient local para que un visualizador arrancado más tarde lo reciba; el perfil predefinido «sensor data» recoge el primer caso, mientras que la durabilidad transient local del mapa no figura en ningún perfil predefinido —todos son volatile— y hay que declararla explícitamente (docs.ros.org, About Quality of Service settings). La regla operativa que hay que dejar escrita en la pizarra: un publicador y un suscriptor solo se conectan si sus políticas son compatibles; una pareja reliable/best effort incompatible no da error, simplemente no fluye ningún mensaje, y ese es el fallo silencioso que más tiempo consume en los proyectos de los estudiantes.

Sesión S33 (jue 26 nov, 1 h), Servicios, acciones, parámetros y launch; colcon

Guion de la clase

0-5 min Recuperación de S32

- Preguntas orales sobre el dibujo del grafo: ¿qué QoS daríais a /scan y por qué? ¿Qué pasa si publicador y suscriptor tienen QoS incompatibles?

5-20 min Servicios y acciones

- Motivar con el límite de los topics: son flujo continuo sin respuesta; para peticiones puntuales con respuesta existe el servicio, modelo llamada-y-respuesta con tipos request/response (docs.ros.org, tutorial Understanding services); ejemplo: consultar el estado de una pinza.
- Acciones: para tareas largas con realimentación; estructura goal + feedback + result, construida sobre topics y servicios, con posibilidad de cancelación (docs.ros.org, tutorial Understanding actions); ejemplo canónico: «navega hasta la pose X», que usará Nav2 en S36 (NavigateToPose).
- Dibujar el diagrama cliente-servidor de una acción con sus tres canales; regla de elección que enunciar: continuo → topic; instantáneo con respuesta → servicio; prolongado, cancelable y con progreso → acción.
- Demos: ros2 service list, ros2 service call /clear std_srvs/srv/Empty con turtlesim; ros2 action list y ros2 action send_goal con /rotate_absolute de turtlesim.

20-30 min Parámetros y launch files

- Parámetros: valores de configuración propios de cada nodo, modificables en caliente (ros2 param list, ros2 param set) y cargables desde YAML (docs.ros.org, tutorial Understanding parameters); conectar con el proyecto: los cientos de parámetros de Nav2 que tocarán en S36 son exactamente esto.
- Launch files: un sistema real arranca decenas de nodos con remapeos y parámetros; un launch file en Python los describe y lanza juntos (docs.ros.org, tutorial Creating a launch file); mostrar un ejemplo mínimo de 15 líneas con dos nodos.

30-50 min Workspaces y paquetes con colcon

- Presentar la unidad de distribución: el paquete, con su package.xml (metadatos y dependencias) y su definición de build ament (docs.ros.org, tutorial Creating a package).
- Estructura de un workspace: src/ con los paquetes; colcon build genera build/, install/ y log/; tras construir, source install/setup.bash superpone (overlay) el workspace sobre la instalación base (underlay) (docs.ros.org, tutorial Creating a workspace).
- Escribir en la pizarra la secuencia completa que mañana ejecutarán: mkdir -p ~/ros2_ws/src → cd ~/ros2_ws/src → ros2 pkg create --build-type ament_python mi_paquete → cd ~/ros2_ws → colcon build --symlink-install → source install/setup.bash → ros2 run mi_paquete mi_nodo.
- Advertencias de veterano: no mezclar terminales con y sin source; no hacer source del install dentro de la terminal donde se construye por primera vez; --symlink-install permite editar Python sin reconstruir.

50-60 min Cierre y preparación del taller

- Repasar la checklist del taller de mañana: ROS 2 Jazzy funcionando (talker/listener), Gazebo instalado (gz sim), paquetes ros-jazzy-ros-gz y ros-jazzy-turtlebot3* o el contenedor docker del curso.
- Cuestionario relámpago de 3 preguntas (topic/servicio/acción) con mando o formulario para fijar la regla de elección.

33.1. Servicios y acciones: comunicación con semántica de tarea

El modelo publicación/suscripción cubre los flujos de datos, pero no toda interacción es un flujo. Cuando un nodo necesita pedir algo a otro y esperar el resultado, «dame el mapa», «abre la pinza», «resetea el filtro», el patrón adecuado es el servicio: una llamada con mensaje de petición y mensaje de respuesta, definidos juntos en un fichero .srv; a diferencia de los topics, que sirven a nodos que quieren datos continuamente, los servicios «solo proporcionan datos cuando un cliente los solicita específicamente» (docs.ros.org, tutorial Understanding services). El servicio es sin embargo inadecuado para tareas prolongadas: la llamada bloquea la lógica del cliente y no ofrece ni progreso ni cancelación. Para eso ROS 2 define las acciones: «están pensadas para tareas de larga duración; constan de tres partes: un objetivo (goal), realimentación (feedback) y un resultado (result)», se construyen internamente sobre topics y servicios, y pueden cancelarse durante la ejecución (docs.ros.org, tutorial Understanding actions). La acción es la interfaz natural de la robótica de tarea: «navega hasta esta pose» (la acción NavigateToPose de Nav2, S36) o «mueve el brazo a esta configuración» (MoveGroup en MoveIt 2, S37) son acciones, con el feedback alimentando la interfaz de usuario y la cancelación conectada al botón de parada.

La regla de elección que los estudiantes deben interiorizar, porque diseñar interfaces es diseñar el sistema: datos que fluyen continuamente y admiten pérdida → topic; petición instantánea con respuesta → servicio; tarea prolongada, cancelable y con progreso → acción. En la corrección de proyectos de años anteriores, el error de diseño más común es implementar como topic con «flags» lo que semánticamente es una acción, perdiendo cancelación y resultado; merece la pena decirlo explícitamente.

33.2. Parámetros y launch files: configurar y orquestar

Un parámetro es un valor de configuración asociado a un nodo concreto, la frecuencia de publicación, el nombre del puerto serie, la ganancia de un controlador, tipado y accesible en ejecución: ros2 param list los enumera, ros2 param get y set los leen y modifican en caliente, y ros2 param dump/load los vuelca y recupera en YAML (docs.ros.org, tutorial Understanding parameters). El diseño es deliberadamente distinto de las variables globales de ROS 1: en ROS 2 cada parámetro pertenece a un nodo, lo que hace el sistema componible. La consecuencia práctica aparecerá en S36 y S37: «configurar Nav2» o «configurar MoveIt 2» significa editar árboles YAML de parámetros de decenas de nodos, y entender este mecanismo ahora evita que entonces parezca magia.

Los launch files responden a la pregunta operativa: si mi robot son treinta nodos con remapeos, parámetros y namespaces, ¿cómo lo arranco? Un launch file, en ROS 2, típicamente un script Python que construye un LaunchDescription, declara qué nodos ejecutar, con qué parámetros y remapeos, y permite composición jerárquica: el launch del robot incluye el del driver, el de la localización y el de la navegación (docs.ros.org, tutorial Creating a launch file). Conviene presentar el launch file como lo que es conceptualmente: la descripción ejecutable de la arquitectura del sistema, el equivalente software del esquema eléctrico del armario en el bloque 3.

33.3. Workspaces, paquetes y colcon

La unidad de organización y distribución del código es el paquete: un directorio con un fichero package.xml que declara metadatos y dependencias, más la información de construcción del sistema ament, setup.py y setup.cfg en paquetes Python, CMakeLists.txt en C++, (docs.ros.org, tutorial Creating a package). Los paquetes viven en workspaces: un directorio con una carpeta src/ que contiene los paquetes fuente. La herramienta colcon construye todos los paquetes del workspace respetando su grafo de dependencias y genera los directorios build/, install/ y log/; el fichero install/setup.bash, al hacerle source, superpone el workspace propio (overlay) sobre la instalación

base de ROS 2 (underlay), de modo que los paquetes propios y sus ejecutables pasan a ser visibles para `ros2 run` y `ros2 launch` (docs.ros.org, tutoriales [Creating a workspace](#) y [Using colcon to build packages](#)). Para paquetes Python, la opción `--symlink-install` instala enlaces simbólicos en lugar de copias, lo que permite editar el código y volver a ejecutar sin reconstruir, la configuración que usaremos en el taller.

El modelo underlay/overlay merece dos minutos de teoría porque explica la familia entera de errores «package not found»: cada terminal tiene su propio entorno; una terminal donde no se ha hecho `source` del overlay no ve los paquetes propios, y una donde se hizo `source` de un `install` obsoleto ve versiones viejas. La disciplina que impondremos en el laboratorio: una terminal para construir (sin `source` del overlay la primera vez) y terminales de ejecución donde se hace `source` tras cada `build` que cambie la estructura del paquete.

Sesión S34 (vie 27 nov, 2 h), Taller: rclpy, Gazebo y TF2

Guion de la clase

0-10 min Arranque y verificación de entornos

- Pasar lista de entornos: cada pareja ejecuta `ros2 run demo_nodes_cpp talker` y `ros2 run demo_nodes_py listener`; los entornos rotos migran al contenedor docker del curso mientras el resto avanza.
- Presentar el objetivo del taller en la pizarra: (1) paquete propio con publicador y suscriptor en rclpy; (2) TurtleBot 3 simulado en Gazebo teleoperado y observado; (3) lectura del árbol TF2 conectándolo con las poses del bloque 4.

10-45 min Parte 1, Paquete Python con publicador y suscriptor

- Crear el workspace y el paquete, comandos dictados y proyectados: `mkdir -p ~/ros2_ws/src` && `cd ~/ros2_ws/src`, después `ros2 pkg create --build-type ament_python --license Apache-2.0 curso_pubsub` (docs.ros.org, tutorial Creating a package).
- Escribir el publicador mínimo siguiendo el tutorial oficial Writing a simple publisher and subscriber (Python) (docs.ros.org): clase que hereda de Node, `create_publisher(String, 'topic', 10)`, timer a 2 Hz con `create_timer` y callback que hace publish; comentar línea a línea qué es `rclpy.init`, `rclpy.spin` y por qué el nodo no termina.
- Registrar el entry point en `setup.py` (console_scripts), añadir dependencias a `package.xml` (rclpy, std_msgs) y construir: `cd ~/ros2_ws` && `colcon build --symlink-install`.
- En una segunda terminal: `source install/setup.bash` y `ros2 run curso_pubsub talker`; verificar con `ros2 topic echo /topic` y `ros2 topic hz /topic`.
- Escribir el suscriptor (`create_subscription` con callback que loguea), reconstruir y ejecutar el par completo; extensión para parejas rápidas: publicar `geometry_msgs/msg/Twist` y cambiar la frecuencia con un parámetro declarado con `declare_parameter`.

45-55 min Descanso (10 min)

- Pausa.

55-85 min Parte 2, Simulación con Gazebo

- Dos ideas de teoría antes de tocar nada (5 min): Gazebo es un simulador de dinámica de cuerpo rígido con sensores simulados, proceso independiente de ROS; el mundo y el robot se describen en SDF/URDF; y el puente `ros_gz_bridge` traduce entre topics de Gazebo y topics de ROS 2 (documentación oficial de Gazebo, gazebo-sim.org/docs, y del paquete `ros_gz`).
- Lanzar la simulación del TurtleBot 3 en Gazebo (mundo `turtlebot3_world`) con el launch proporcionado por el curso; identificar el proceso `gz sim` y los nodos puente con `ros2 node list`.
- Inventario de interfaces: `ros2 topic list -t`, localizar `/scan` (`sensor_msgs/LaserScan`), `/odom` (`nav_msgs/Odometry`), `/cmd_vel` (`geometry_msgs/Twist`); comprobar frecuencias con `ros2 topic hz /scan`.
- Teleoperar: `ros2 run teleop_twist_keyboard teleop_twist_keyboard` y observar en paralelo `ros2 topic echo /cmd_vel`; conectar con B4: el Twist es exactamente la velocidad (v , ω) del modelo diferencial.
- Cerrar el bucle conceptual: sustituir el teleop por un nodo propio que publique `/cmd_vel` a 10 Hz y haga avanzar el robot, es el mismo publicador de la Parte 1 con otro tipo de mensaje; el robot simulado se mueve con software escrito por ellos.
- Visualizar `/scan` en RViz2 con Fixed Frame `odom`; señalar que RViz no simula: solo dibuja lo que fluye por los topics.

85-110 min Parte 3, El árbol de transformadas TF2

- Teoría en pizarra (5 min): un robot es un conjunto de sistemas de referencia (map, odom, base_link, base_scan...); TF2 mantiene «la relación entre sistemas de coordenadas en una estructura de árbol almacenada en el tiempo» y permite transformar puntos y poses entre dos frames cualesquiera en cualquier instante (docs.ros.org, tutorial Introducing tf2 y concepto About tf2); conectar explícitamente con la composición de poses del bloque 4: cada arista del árbol es una transformada homogénea con sello de tiempo.
- Generar y abrir el árbol del TurtleBot: `ros2 run tf2_tools view_frames` (produce frames.pdf); identificar la cadena odom → base_footprint → base_link → base_scan y quién publica cada arista (la odometría publica odom→base, robot_state_publisher publica las aristas fijas desde el URDF).
- Consultar una transformada en vivo: `ros2 run tf2_ros tf2_echo odom base_link` mientras el robot se mueve; verificar que coincide con /odom.
- Pregunta de conexión con B6 para dejar sembrada: ¿quién publicará map → odom cuando haya localización? (AMCL, en S36; la corrección de deriva es exactamente esa arista).

110-120 min Cierre

- Recogida de evidencias del taller: captura de rqt_graph con el grafo completo y el frames.pdf, que se suben al campus como entregable ligero.
- Anunciar S35: pasamos de mover el robot a decidir por dónde, planificación con mapas de ocupación, A*, PRM y RRT; lectura opcional: Corke (2023), cap. 5, pp. 177-187.

34.1. Anatomía de un nodo rclpy

rclpy es la biblioteca cliente Python de ROS 2; su equivalente C++ es rclcpp, y ambas son fachadas sobre la misma capa común rcl, lo que garantiza que un nodo Python y uno C++ tengan la misma semántica de comunicación. El patrón canónico que seguimos en el taller es el del tutorial oficial (docs.ros.org, Writing a simple publisher and subscriber (Python)): una clase que hereda de Node fija el nombre del nodo en el constructor y crea allí sus recursos, create_publisher(tipo, nombre_topic, profundidad_QoS), create_subscription(tipo, nombre, callback, profundidad), create_timer(periodo, callback); el programa principal llama a rclpy.init(), instancia el nodo y cede el control a rclpy.spin(nodo), que procesa callbacks indefinidamente. Conviene detenerse en el modelo de ejecución, porque es donde los estudiantes de formación no informática tropiezan: el nodo no «ejecuta un bucle» visible; declara callbacks y un ejecutor los invoca cuando llegan mensajes o vencen temporizadores, el mismo modelo dirigido por eventos de las interrupciones del bloque 3, elevado de la ISR al proceso.

El empaquetado es parte del contenido, no burocracia: el entry point declarado en setup.py bajo console_scripts es lo que convierte una función Python en un ejecutable invocable con `ros2 run`, y las dependencias declaradas en package.xml son las que `colcon` y las herramientas de despliegue usan para reproducir el entorno. La prueba de madurez del taller es que, al final de la Parte 1, cada pareja tiene un paquete que cualquier otra pareja puede clonar, construir con `colcon build` y ejecutar, código robótico como producto reproducible, no como script suelto.

34.2. Simulación con Gazebo y el puente ros_gz

Gazebo (la serie moderna, antes llamada Ignition) es un simulador de robots de propósito general: integra un motor de física de cuerpo rígido con contactos y fricción, renderizado para cámaras simuladas y una biblioteca de sensores, LiDAR, IMU, cámaras RGB y de profundidad, con modelos de ruido configurables (documentación oficial de Gazebo, gazebosim.org/docs). Mundos y robots se describen en SDF, y los robots ROS habitualmente en URDF, el formato de descripción cinemática que

TF2 y MoveIt 2 también consumen. El punto arquitectónico que importa entender es que Gazebo no es parte de ROS: es un proceso independiente con su propio middleware de topics (gz-transport), y la integración se hace con el paquete `ros_gz`, cuyo componente `ros_gz_bridge` traduce mensajes entre ambos mundos topic a topic con tipos mapeados (documentación del paquete `ros_gz`). Para el estudiante, la consecuencia es la que anunciamos en S32: una vez el puente publica `/scan`, `/odom` y suscribe `/cmd_vel`, el resto del grafo, teleoperación, sus nodos, mañana Nav2, no puede distinguir la simulación del robot real.

Metodológicamente, la simulación es la herramienta que hace posible el proyecto integrador en el calendario de la asignatura: permite desarrollar y probar el stack completo de navegación sin turno de laboratorio, reservando el robot físico para la validación final. Conviene también nombrar sus límites con honestidad de ingeniero: el contacto y la fricción simulados son aproximaciones, los sensores simulados son «demasiado perfectos» incluso con ruido gaussiano añadido, y el salto sim-to-real, que en el bloque 8 reaparecerá como problema central del aprendizaje por refuerzo, empieza exactamente aquí.

34.3. El árbol de transformadas TF2

TF2 es la respuesta de ROS 2 a una necesidad que el bloque 4 dejó planteada: en un robot todo dato geométrico está expresado en algún sistema de referencia, el barrido láser en `base_scan`, la odometría en `odom`, la meta de navegación en `map`, y componer poses a mano, con marcas de tiempo distintas, es inviable a escala de sistema. TF2 «mantiene la relación entre sistemas de coordenadas en una estructura de árbol almacenada en el tiempo (buffered in time) y permite al usuario transformar puntos, vectores, etc., entre dos sistemas de coordenadas cualesquiera en cualquier instante deseado» (docs.ros.org, tutorial *Introducing tf2*). Cada arista del árbol es una transformada homogénea con sello de tiempo, exactamente los elementos de $SE(3)$ del bloque 4, publicada en los topics `/tf` (aristas dinámicas) y `/tf_static` (aristas fijas); consultar la transformada entre dos frames es componer la cadena de aristas que los une, con interpolación temporal en las dinámicas. La convención de frames de un robot móvil, estandarizada por la comunidad (REP 105), es la cadena `map` → `odom` → `base_link`: `odom` → `base_link` la publica la odometría, continua pero con deriva; `map` → `odom` la publica el sistema de localización, precisamente para corregir esa deriva, cuando en S36 activemos AMCL, el filtro de partículas del bloque 6 se materializará como el publicador de esa arista.

Las herramientas de inspección cierran el círculo práctico: `ros2 run tf2_tools view_frames` escucha unos segundos y genera un PDF con el árbol completo, quién publica cada transformada y a qué frecuencia, y `ros2 run tf2_ros tf2_echo frame_origen frame_destino` imprime en vivo la transformada compuesta (docs.ros.org, tutoriales de *tf2*). El error conceptual a vigilar en el taller: el árbol es árbol, no grafo, cada frame tiene un único padre, y dos fuentes publicando la misma arista (por ejemplo, dos odometrías sobre `odom` → `base_link`) producen los saltos erráticos que los estudiantes describirán como «el robot tiembla en RViz».

Sesión S35 (mié 2 dic, 1 h), Planificación de trayectorias: mapas, A*, PRM y RRT

Guion de la clase

0-5 min Formulación del problema

- Definir con Lynch y Park: «la planificación de movimiento es el problema de encontrar un movimiento del robot desde un estado inicial hasta un estado meta que evite los obstáculos del entorno y satisfaga otras restricciones» (Lynch y Park, 2017, p. 353); recordar el C-space del bloque 2 y definir C_{free} .
- Distinguir planificación de caminos (subproblema puramente geométrico, el «problema del transportista de pianos») de planificación de movimiento completa (Lynch y Park, 2017, p. 355); hoy hacemos caminos; la restricción no holonómica del coche ($n=3$, $m=2$, «no puede deslizarse lateralmente a una plaza de aparcamiento», p. 355) explica por qué a veces no basta.
- Vocabulario de propiedades para comparar todo lo que sigue: planificador completo, completo en resolución y probabilísticamente completo (Lynch y Park, 2017, p. 356); anotar los tres en una esquina de la pizarra y volver a ellos con cada algoritmo.

5-15 min Mapas de ocupación y de coste

- Ocupación: «array de celdas, típicamente cuadradas», cada una con la transitabilidad de esa celda; en el caso binario 1 = obstáculo, 0 = libre (Corke, 2023, p. 177); mostrar la figura 5.6 con las dos representaciones de mapa, grafo y rejilla (Corke, 2023, p. 166).
- Origen probabilístico (enlace con B6): el mapa como «campo de variables aleatorias binarias en rejilla regular» estimado con log-odds por el algoritmo `occupancy_grid_mapping` (Thrun et al., 2005, pp. 284-286); su utilidad principal es el posprocesado tras SLAM para obtener mapas aptos para planificar (Thrun et al., 2005, p. 284).
- Truco esencial: inflar los obstáculos el radio del robot para planificar con un robot puntual (Corke, 2023, p. 181); es la misma idea que los C-obstáculos de Lynch y Park (2017, p. 359) y reaparecerá como inflation layer del costmap en Nav2.
- Del binario al coste: D^* generaliza la rejilla a un mapa de coste con c real positivo por celda, $c\sqrt{2}$ en diagonal e infinito en obstáculos (Corke, 2023, p. 182), coste = tiempo, rugosidad o riesgo.

15-30 min Búsqueda en grafos: de UCS a A*

- Convertir la rejilla en grafo (8 vecinos) y buscar: presentar frontera y explorados con UCS eligiendo el menor coste acumulado (cost-to-come) con cola de prioridad (Corke, 2023, pp. 172-173).
- A^* : expandir según $f(v) = g(v) + h(v)$, coste acumulado más heurística del coste restante (Corke, 2023, p. 175); estructura OPEN/CLOSED y `past_cost` en el pseudocódigo de Lynch y Park (2017, pp. 365-366).
- Enunciar el teorema clave: « A^* garantiza devolver el camino de menor coste solo si la heurística es admisible, es decir, si no sobreestima» (Corke, 2023, p. 175); euclídea admisible; $h = 0$ degenera en UCS y explora todo.
- Nota histórica de 30 segundos: el «truco de A^* » se publicó en 1968 por Hart, Nilsson y Raphael, del equipo del robot Shakey (Corke, 2023, p. 174).
- Dijkstra en una frase: coste que se expande desde el origen, «análogo a la transformada de distancia o planificador de frente de onda» (Corke, 2023, p. 199); mencionar la transformada de distancia sobre la rejilla (Corke, 2023, pp. 177-181) como el mismo concepto sin heurística.
- D^* para replanificar: extensión de A^* que «permite modificar el mapa en cualquier momento mientras el robot se mueve» reutilizando el plan (Corke, 2023, p. 184); en el ejemplo del libro, replanificar cuesta $\approx 12\%$ de la planificación original (Corke, 2023, p. 184).

30-45 min Métodos de muestreo: PRM y RRT

- Motivación doble: coste de la fase de planificación en rejilla frente a consultas baratas (Corke, 2023, p. 184) y maldición de la dimensión, los métodos de rejilla son «imprácticos más allá de unos pocos grados de libertad» (Lynch y Park, 2017, p. 378); un brazo de 6 gdl no cabe en una rejilla.
- PRM: muestrear N configuraciones libres, conectar vecinos con caminos rectos libres de colisión y obtener un roadmap independiente de inicio y meta (Corke, 2023, pp. 185-186); consulta: conectar q_{start} y q_{goal} al grafo y buscar con A^* (Lynch y Park, 2017, p. 384); multiconsulta por construcción.
- Enseñar el fallo típico con la fig. 5.19: roadmap mal conectado con 50 vértices frente a bien conectado con 300 (Corke, 2023, p. 187); propiedad: probabilísticamente completo, no óptimo.
- RRT: recorrer el Algoritmo 10.3 línea a línea, muestrear x_{samp} , buscar $x_{nearest}$ en el árbol, planificador local hacia x_{new} , añadir si no colisiona, éxito al tocar la región meta (Lynch y Park, 2017, p. 379); intuición: las muestras uniformes «tiran» del árbol y lo hacen explorar rápidamente C_{free} (Lynch y Park, 2017, p. 379).
- RRT con cinemática: la versión de Corke planifica el problema del piano con primitivas de Dubins respetando el modelo de movimiento del vehículo (Corke, 2023, pp. 196-199), así se planifica para el coche no holonómico.
- Variantes en un minuto: RRT bidireccional (dos árboles que se encuentran) y RRT*, que «recablea el árbol continuamente» y tiende al óptimo (Lynch y Park, 2017, pp. 382-383); referencias originales LaValle y Karaman et al. (Corke, 2023, p. 202).

45-55 min Campos potenciales y ejercicio de síntesis

- Campos potenciales: meta = potencial bajo, obstáculos = potencial alto, el robot desciende el gradiente; es control reactivo en tiempo real más que planificación, y su defecto son los mínimos locales (Lynch y Park, 2017, p. 386, y fig. 10.21 en p. 388).
- Ejercicio en parejas (7 min): tabla comparativa en la pizarra, para transformada de distancia/ A^* , D^* , PRM, RRT y potenciales: ¿completo?, ¿óptimo?, ¿coste de planificación frente a consulta?, ¿sirve para 6 gdl?, ¿sirve con mapa cambiante?, rellenarla entre todos.

55-60 min Cierre

- Anticipar S36: Nav2 empaqueta exactamente esto, costmaps con inflado, un planificador global tipo A^* /Smac sobre el costmap y un controlador local, y lo orquesta con behavior trees.

35.1. Formulación: del mapa al espacio de configuraciones

La planificación de movimiento es, en la definición de Lynch y Park, «el problema de encontrar un movimiento del robot desde un estado inicial hasta un estado meta que evite los obstáculos del entorno y satisfaga otras restricciones, como límites articulares o de par» (Lynch y Park, 2017, p. 353). El escenario natural del problema es el espacio de configuraciones que definimos en el bloque 2: cada punto del C-space es una configuración q del robot, y el subconjunto libre C_{free} reúne las configuraciones sin colisión ni violación de límites (Lynch y Park, 2017, p. 353). Dentro del problema general conviene aislar el subproblema que hoy resolvemos: la planificación de caminos es «el problema puramente geométrico de encontrar un camino libre de colisión $q(s)$, $s \in [0,1]$ » entre configuración inicial y final, sin atender a dinámica, duración ni restricciones de actuación, «a veces llamado el problema del transportista de pianos» (Lynch y Park, 2017, p. 355). No siempre basta: un coche tiene $n = 3$ grados de libertad de configuración pero $m = 2$ entradas de control y «no puede deslizarse lateralmente a una plaza de aparcamiento» (Lynch y Park, 2017, p. 355), la restricción no holonómica del bloque 2 reaparece aquí como restricción sobre qué caminos son ejecutables, y motivará los planificadores con modelo de movimiento del final de la sesión.

Para comparar planificadores necesitamos vocabulario de propiedades (Lynch y Park, 2017, pp. 355-356): un planificador es completo si garantiza encontrar solución en tiempo finito cuando existe y

reportar fallo cuando no; es completo en resolución si lo garantiza a la resolución de la discretización elegida; y es probabilísticamente completo si la probabilidad de encontrar solución, cuando existe, tiende a 1 al crecer el tiempo de planificación. A ello se suman la optimalidad (¿minimiza el coste o solo satisface?), la distinción entre planificadores multiconsulta y de consulta única, y la complejidad computacional (Lynch y Park, 2017, pp. 355-356). Estas etiquetas, que iremos asignando a cada algoritmo, son las que un ingeniero usa para elegir planificador en un proyecto real, y las que Nav2 y MoveIt 2 dan por sabidas en sus páginas de configuración.

35.2. Mapas de ocupación y mapas de coste

El soporte de datos dominante en robótica móvil es el mapa de ocupación: «un array de celdas, típicamente cuadradas, donde cada celda contiene información sobre la transitabilidad de esa celda»; en el caso más simple la celda toma dos valores, uno si está ocupada, es un obstáculo, y cero si está libre (Corke, 2023, p. 177). Frente a la representación alternativa de grafo, vértices que son lugares y aristas que son rutas, la rejilla de ocupación discretiza el espacio de forma uniforme (Corke, 2023, p. 166, fig. 5.6). Su fundamento probabilístico lo dimos en el bloque 6 y conviene reactivarlo: para Thrun, Burgard y Fox la idea básica es «representar el mapa como un campo de variables aleatorias dispuestas en una rejilla regular», cada una binaria, y estimar su posterior $p(m \mid z_{1:t}, x_{1:t})$ con poses conocidas; el algoritmo `occupancy_grid_mapping` actualiza cada celda observada en representación log-odds, que evita inestabilidades numéricas cerca de 0 y 1 (Thrun et al., 2005, pp. 284-286). El propio libro sitúa su papel en el pipeline que nuestros estudiantes montarán: muchas técnicas de SLAM no generan mapas «aptos para planificación de caminos y navegación», y las rejillas de ocupación se usan como posprocesado una vez resuelto el SLAM (Thrun et al., 2005, p. 284), exactamente la relación entre el mapa que guardaron en B6 y lo que hoy hacemos con él.

Dos transformaciones convierten el mapa crudo en un soporte de planificación. La primera es el inflado de obstáculos: para poder tratar el robot como un punto, se expanden los obstáculos el radio del robot, una dilatación morfológica, en términos de procesamiento de imagen, de modo que cualquier celda libre del mapa inflado es alcanzable por el robot completo (Corke, 2023, p. 181). Es el mismo movimiento conceptual que Lynch y Park hacen en el C-space: para un robot móvil circular, «crecer» los obstáculos el radio del robot convierte el problema en el de un punto entre C-obstáculos (Lynch y Park, 2017, p. 359). La segunda es pasar de ocupación binaria a coste: D^* «generaliza la rejilla de ocupación a un mapa de coste que representa el coste $c \in \mathbb{R}$, $c > 0$, de atravesar cada celda en dirección horizontal o vertical», con coste $c \cdot \sqrt{2}$ en diagonal e infinito en los obstáculos; el coste puede codificar tiempo de travesía, rugosidad del terreno o confort (Corke, 2023, p. 182). Cuando en S36 abramos el costmap de Nav2, capa estática, capa de obstáculos, capa de inflado, los estudiantes deben reconocer ambas transformaciones institucionalizadas en software.

35.3. Búsqueda en grafos: de la transformada de distancia a A^*

Sobre una rejilla con vecindad de 8, planificar es buscar en un grafo. La familia clásica se organiza por lo que se propaga desde dónde. La transformada de distancia asigna a cada celda libre su distancia al objetivo, euclídea o Manhattan, computada de forma iterativa; seguir el gradiente descendente de esa distancia desde cualquier celda produce el camino más corto al objetivo (Corke, 2023, pp. 177-179). Es un planificador completo y óptimo sobre la rejilla, con un coste de planificación alto que se paga una vez por objetivo. La búsqueda de coste uniforme (UCS) formaliza la misma propagación en términos de grafo: mantiene una frontera ordenada por coste acumulado desde el origen (cost-to-come) en una cola de prioridad, expande siempre el vértice más barato y reparenta cuando descubre un camino mejor (Corke, 2023, pp. 172-173). El algoritmo de Dijkstra pertenece a esta familia: el coste computado «se expande hacia fuera desde el vértice inicial y es análogo a la transformada de distancia o planificador de frente de onda» (Corke, 2023, p. 199).

A* añade la pieza que hace la búsqueda dirigida: expandir según $f(v) = g(v) + h(v)$, la suma del coste acumulado, el de UCS, y una heurística que estima el coste restante hasta la meta (Corke, 2023, p. 175). En el pseudocódigo de Lynch y Park, el algoritmo mantiene la lista OPEN ordenada por coste total estimado, la lista CLOSED de nodos ya explorados, el array `past_cost` con el mejor coste conocido hasta cada nodo y los punteros `parent` que reconstruyen el camino (Lynch y Park, 2017, pp. 365-366). La garantía central debe quedar enunciada con precisión: «A* garantiza devolver el camino de menor coste solo si la heurística es admisible, es decir, si no sobreestima el coste de alcanzar la meta»; la distancia euclídea es admisible por ser el mínimo coste posible, $h = 0$ es admisible pero degenera en UCS visitando todos los vértices, y una heurística no admisible puede producir caminos subóptimos (Corke, 2023, p. 175). La anécdota fundacional ancla la historia: el «truco de A*» lo publicaron en 1968 Hart, Nilsson y Raphael, miembros del equipo del robot Shakey en el Stanford Research Institute (Corke, 2023, p. 174). Sobre estas ideas, D*, «A* dinámico», aporta lo que el robot móvil necesita: replanificación eficiente cuando los costes cambian; «D* permite hacer cambios en el mapa en cualquier momento mientras el robot está en movimiento», reutilizando el trabajo previo, y en el ejemplo del libro la replanificación tras descubrir una región de coste alto expande en torno al 12 % de los vértices de la planificación original (Corke, 2023, pp. 182-184).

35.4. Métodos de muestreo: PRM y RRT

Dos límites empujan más allá de la rejilla. El primero es económico: la transformada de distancia y D* concentran el gasto en la fase de planificación para una consulta barata, pero el plan depende del objetivo, si el objetivo cambia hay que replanificar entero (Corke, 2023, p. 184). El segundo es dimensional: los métodos de rejilla son óptimos a su resolución, pero su complejidad los hace «imprácticos más allá de unos pocos grados de libertad» (Lynch y Park, 2017, p. 378), la rejilla de un brazo de 6 gdl a resolución modesta tiene más celdas de las que caben en memoria. Los métodos de muestreo renuncian «a las soluciones óptimas en resolución a cambio de la capacidad de encontrar soluciones satisfactorias rápidamente en espacios de alta dimensión» (Lynch y Park, 2017, p. 378): muestrean el C-space, comprueban colisión solo en las muestras y sus conexiones, y construyen un grafo o árbol de movimientos factibles.

El roadmap probabilístico (PRM) es el representante multiconsulta. El coste computacional de la transformada de distancia y la esqueletonización llevó «al desarrollo de métodos probabilísticos que muestrean el mapa de forma dispersa, el más conocido de los cuales es el probabilistic roadmap o PRM» (Corke, 2023, p. 185). La fase de planificación genera N configuraciones aleatorias, descarta las que caen en obstáculos e intenta conectar cada una con sus vecinas mediante caminos rectos libres de colisión, comprobados por muestreo a lo largo del segmento (Corke, 2023, p. 186); el resultado es un grafo no dirigido, el roadmap, independiente del inicio y de la meta (Corke, 2023, p. 185). En la fase de consulta se conectan `q_start` y `q_goal` a sus nodos más próximos y se busca el camino en el grafo, «típicamente usando A*» (Lynch y Park, 2017, p. 384); construido el roadmap, cada consulta adicional es casi gratis, que es la definición operativa de planificador multiconsulta (Lynch y Park, 2017, p. 384). Sus modos de fallo son instructivos: con pocas muestras el roadmap queda fragmentado en componentes desconexas, la figura del libro contrasta 50 vértices mal conectados con 300 bien conectados (Corke, 2023, pp. 187-188), y los pasajes estrechos, poco probables de muestrear, son su talón de Aquiles; el planificador es probabilísticamente completo, no completo ni óptimo.

El árbol aleatorio de exploración rápida (RRT) es el representante de consulta única y el que mejor tolera restricciones de movimiento. El algoritmo básico (Lynch y Park, 2017, p. 379, Algoritmo 10.3) crece un árbol desde el estado inicial repitiendo cuatro pasos: muestrear `x_samp` del espacio (casi uniforme, con un ligero sesgo hacia la meta), localizar el nodo más cercano `x_nearest` del árbol, aplicar un planificador local que avance desde `x_nearest` una distancia pequeña hacia `x_samp` produciendo `x_new`, y añadir `x_new` al árbol si el movimiento está libre de colisión, terminando con éxito al alcanzar

la región meta. La intuición del nombre está en la misma página: las muestras distribuidas por todo el espacio «tiran» del árbol y provocan que explore rápidamente C_{free} (Lynch y Park, 2017, p. 379). La elección del planificador local es el punto de acoplamiento con la cinemática: puede ser una recta para sistemas sin restricciones, controles discretizados integrados en el tiempo para sistemas con dinámica, o curvas de Reeds-Shepp para vehículos con radio de giro (Lynch y Park, 2017, p. 382); la implementación que muestra Corke resuelve el problema del plano no holonómico conectando configuraciones (x, y, θ) con primitivas de Dubins que respetan el modelo de movimiento del vehículo, y usa el polígono del vehículo para la comprobación de colisiones (Corke, 2023, pp. 196-198). Dos variantes cierran el mapa (Lynch y Park, 2017, pp. 382-383): el RRT bidireccional crece un árbol desde el inicio y otro desde la meta e intenta conectarlos, y RRT* «recablea continuamente el árbol de búsqueda para asegurar que siempre codifica el camino más corto desde el inicio hasta cada nodo», recuperando asintóticamente la optimalidad que el RRT básico no tiene. Las referencias originales, para el estudiante que quiera profundizar: el RRT se describe en LaValle (1998, 2001) y RRT* en Karaman et al. (Corke, 2023, p. 202).

35.5. Campos potenciales: planificar sin plan

La última familia cambia de naturaleza: en los campos potenciales virtuales «a la configuración meta se le asigna un potencial virtual bajo y a los obstáculos un potencial virtual alto», y el robot se mueve bajo una fuerza proporcional al gradiente negativo del potencial, que «lo empuja de forma natural hacia la meta y lejos de los obstáculos» (Lynch y Park, 2017, p. 386). Como el gradiente se evalúa en tiempo real, el método funciona como control reactivo más que como planificación previa, y con los sensores adecuados «puede incluso manejar obstáculos que se mueven o aparecen inesperadamente» (Lynch y Park, 2017, p. 386). Su defecto es estructural y hay que mostrarlo, no solo enunciarlo: el robot puede quedar atrapado en mínimos locales del potencial lejos de la meta aunque exista camino factible; la figura del libro muestra un potencial con su mínimo global, un mínimo local y cuatro puntos de silla generados por tres obstáculos (Lynch y Park, 2017, pp. 386-388, fig. 10.21). En sistemas reales los potenciales sobreviven como capa reactiva local, evitar el obstáculo imprevisto, por debajo de un planificador global que garantiza la llegada; esa división del trabajo global/local es exactamente la arquitectura planner/controller de Nav2 que veremos mañana.

Sesión S36 (jue 3 dic, 1 h), Navegación autónoma con Nav2

Guion de la clase

0-5 min Recuperación de S35

- Repaso con la tabla comparativa de planificadores: ¿cuál elegirías para un AMR en un almacén con carretillas en movimiento, y por qué necesita dos niveles (global + local)?

5-20 min Arquitectura de Nav2

- Presentar Nav2 como el stack de navegación de ROS 2: toma una meta y produce `/cmd_vel` combinando planificación, control, costmaps y recuperación; su interfaz externa son acciones (NavigateToPose) (docs.nav2.org, sección Navigation Concepts).
- Dibujar el diagrama de servidores sobre el grafo de S32: `bt_navigator` (orquestador), `planner server` (camino global sobre el costmap global), `controller server` (sigue el camino y produce `/cmd_vel` con el costmap local), `behavior server` (recuperaciones: girar, retroceder, esperar), `smoother server`; cada uno es un nodo con plugins configurables (docs.nav2.org, Navigation Concepts y páginas de configuración de cada servidor).
- Conectar cada servidor con S35: el planner ejecuta algoritmos de la familia A*/Smac o NavFn (Dijkstra) sobre el costmap; el controller es la pieza reactiva local (DWB, MPPI...), la división global/local que motivamos con los campos potenciales.
- Señalar que los servidores son lifecycle nodes gestionados (configure/activate), por eso Nav2 arranca ordenadamente con su launch (docs.nav2.org, Navigation Concepts, Lifecycle Nodes).

20-35 min Costmaps por capas

- Explicar el costmap 2D de Nav2 como materialización del mapa de coste de S35: rejilla con valores de coste que se compone por capas, static layer (el mapa de B6), obstacle/voxel layer (sensores en vivo), inflation layer (gradiente de coste alrededor de obstáculos), (docs.nav2.org, Configuration Guide: Costmap 2D).
- Relacionar inflation layer con el inflado de Corke (2023, p. 181) y con el coste generalizado de D* (Corke, 2023, p. 182): inflar con gradiente en lugar de binario hace que los caminos óptimos se separen de las paredes.
- Global frente a local: costmap global grande y anclado a map para el planner; costmap local pequeño, rodante y anclado a odom para el controller, preguntar a la clase por qué el local NO debe depender de map (respuesta: los saltos de la localización romperían el control reactivo).

35-45 min Behavior trees y recuperación

- Motivar: navegar no es solo planificar; es qué hacer cuando el plan falla; Nav2 orquesta planificación, control y recuperación con un behavior tree ejecutado por `bt_navigator` y descrito en XML (docs.nav2.org, secciones Behavior Trees y Detailed Behavior Tree Walkthrough).
- Recorrer el árbol por defecto de NavigateToPose en proyección: rama principal (calcular camino → seguir camino, con replanificación periódica) y rama de recuperación (limpiar costmaps, girar sobre sí, esperar, retroceder) que se activa al fallar la principal (docs.nav2.org, Detailed Behavior Tree Walkthrough).
- Vender el concepto de BT frente a máquina de estados: modularidad y reutilización, cambiar la política de recuperación es editar XML, no recompilar; los BT reaparecen en el bloque 8 como estructura de agentes.

45-55 min AMCL y demo integrada

- Cerrar el lazo con B6: AMCL es la localización de Nav2, filtro de partículas (MCL adaptativo) sobre el mapa estático que publica la transformada map → odom, la arista que dejamos pendiente en S34 (docs.nav2.org, Configuration Guide: AMCL; fundamento MCL en Thrun et al., 2005, cap. 8).
- Demo (si el tiempo lo permite, en el simulador ya montado): `ros2 launch nav2_bringup tb3_simulation_launch.py`; fijar pose inicial con 2D Pose Estimate en RViz, ver converger la nube de partículas, enviar Nav2 Goal y narrar en vivo qué servidor hace cada cosa (camino verde = planner; robot esquivando = controller; si se atasca = behaviors).
- Mostrar en paralelo `ros2 topic echo /plan` y el árbol TF con la arista map → odom actualizándose.

55-60 min Cierre

- Encargo para S37: instalar MoveIt 2 (ros-jazzy-moveit) o actualizar el contenedor; anunciar que S37 integra el stack completo y cierra con el cuestionario B7 (10 min).

36.1. Nav2: la arquitectura de servidores

Nav2 es el stack de navegación autónoma de ROS 2, heredero del navigation stack de ROS 1: dado un mapa, una localización y una meta, produce las consignas de velocidad que llevan al robot a la meta de forma segura. Su documentación lo describe como un conjunto de servidores modulares e independientes, cada uno un nodo ROS 2 que expone una acción y aloja algoritmos como plugins intercambiables (docs.nav2.org, sección Navigation Concepts). El reparto de papeles reproduce casi literalmente la taxonomía que construimos en S35: el planner server calcula el camino global sobre el costmap global, con plugins como NavFn, basado en Dijkstra/frente de onda, o la familia Smac Planner con A* sobre rejilla y variantes para vehículos con restricciones de giro; el controller server sigue ese camino en el horizonte local, produciendo `/cmd_vel` a alta frecuencia con controladores reactivos (DWB, MPPI, Regulated Pure Pursuit) sobre el costmap local; el behavior server ejecuta comportamientos de recuperación (girar sobre sí mismo, retroceder, esperar, limpiar costmaps); y el smoother server pule los caminos. Por encima, el `bt_navigator` recibe la acción `NavigateToPose` y orquesta a todos los demás (docs.nav2.org, Navigation Concepts y guías de configuración de cada servidor). Los servidores son lifecycle nodes: atraviesan estados gestionados de configuración y activación, lo que da al arranque y parada del stack un orden determinista, un patrón de ingeniería de sistemas que merece señalarse más allá de Nav2 (docs.nav2.org, Navigation Concepts).

Para el estudiante, la lectura importante es que Nav2 no inventa algoritmos: institucionaliza los de S35 detrás de interfaces de plugin, de forma que elegir y ajustar un planificador es una decisión de configuración informada por las propiedades, completitud, optimalidad, coste, tratamiento de la no holonomía, que ya sabemos evaluar. El proyecto integrador pide exactamente eso: justificar la elección de plugins y parámetros de Nav2 para su robot y su entorno, con argumentos de la sesión anterior.

36.2. Costmaps por capas

El costmap 2D de Nav2 es el mapa de coste de S35 convertido en componente de software con actualización continua. Es una rejilla en la que cada celda lleva un coste de travesía, libre, coste creciente, obstáculo letal, y se construye por composición de capas (docs.nav2.org, Configuration Guide: Costmap 2D): la static layer importa el mapa de ocupación construido en el bloque 6; la obstacle layer (o voxel layer en 3D) marca obstáculos con los sensores en vivo, incorporando lo que el mapa estático no sabe, la carretilla aparcada esta mañana; y la inflation layer propaga alrededor de cada obstáculo un gradiente de coste decreciente con la distancia. Esta última capa es la síntesis de dos ideas citadas ayer: el inflado por el radio del robot que permite planificar con un robot puntual (Corke, 2023, p. 181; Lynch y Park, 2017, p. 359) y la generalización de ocupación binaria a coste real de D* (Corke, 2023, p. 182); al inflar con gradiente y no de forma binaria, el planificador global no solo evita

colisiones sino que prefiere caminos alejados de las paredes, y el margen de esa preferencia es un parámetro de diseño (el radio y la pendiente de inflado).

Nav2 mantiene dos costmaps simultáneos, y la diferencia es conceptualmente rica: el costmap global cubre todo el mapa, está anclado al frame map y sirve al planner; el costmap local es una ventana rodante centrada en el robot, anclada al frame odom, y sirve al controller. La elección de frames no es un capricho: la estimación de localización corrige a saltos (la arista map → odom que publica AMCL), y un controlador reactivo que viviera en map vería teletransportarse los obstáculos con cada corrección; en odom el movimiento es continuo aunque derive, la estructura de frames de REP 105 que montamos en S34 resulta ser una decisión de arquitectura de control, no una convención arbitraria.

36.3. Behavior trees, recuperación y el papel de AMCL

La orquestación de Nav2 usa árboles de comportamiento (behavior trees, BT): el bt_navigator carga un árbol descrito en XML cuyos nodos hoja llaman a las acciones de los servidores, y cuya estructura de control, secuencias, alternativas (fallbacks), decoradores de frecuencia, decide qué se intenta, en qué orden y qué se hace al fallar (docs.nav2.org, secciones Behavior Trees y Detailed Behavior Tree Walkthrough). En el árbol por defecto de NavigateToPose, la rama nominal replanifica el camino a frecuencia fija y lo pasa al controlador; cuando algo falla, una rama de recuperación limpia los costmaps, olvidando obstáculos espurios, hace girar el robot para refrescar la percepción, espera o retrocede, y reintenta (docs.nav2.org, Detailed Behavior Tree Walkthrough). Frente a la máquina de estados monolítica, el BT aporta modularidad y legibilidad: cambiar la política de recuperación del robot del proyecto es editar un XML, y el mismo formalismo reaparecerá en el bloque 8 como esqueleto de agentes de decisión más generales.

Queda la pieza que Nav2 asume resuelta: saber dónde está el robot. La proporciona AMCL (Adaptive Monte Carlo Localization), la implementación en Nav2 de la localización de Monte Carlo del bloque 6, un filtro de partículas sobre el mapa estático, con muestreo adaptativo del número de partículas, cuyo fundamento estudiamos en Thrun et al. (2005, cap. 8) (docs.nav2.org, Configuration Guide: AMCL). Su salida arquitectónica es exactamente la arista TF que dejamos pendiente en S34: AMCL publica map → odom, la corrección de la deriva odométrica, mientras la odometría sigue publicando odom → base_link. En la demo final, ver converger la nube de partículas tras fijar la pose inicial en RViz y, a la vez, estabilizarse la transformada map → odom, es la mejor síntesis visual de los bloques 6 y 7 trabajando juntos.

Sesión S37 (vie 4 dic, 2 h), Manipulación con MoveIt 2 e integración del stack

Guion de la clase

0–10 min Recuperación y planteamiento

- Pregunta puente: en S35 dijimos que la rejilla no escala a 6 gdl y que ahí mandan los métodos de muestreo (Lynch y Park, 2017, p. 378); hoy lo comprobamos: planificar para el brazo del laboratorio es planificar en un C-space de dimensión 6.
- Mapa de la sesión: arquitectura de MoveIt 2 → OMPL → planificación cartesiana → taller de integración → cuestionario B7.

10–35 min Arquitectura de MoveIt 2: move_group y planning scene

- Presentar MoveIt 2 como la plataforma de manipulación de ROS 2: planificación de movimiento, ejecución, colisiones, cinemática inversa y percepción del entorno para brazos (moveit.picknik.ai, sección Concepts).
- Dibujar la arquitectura alrededor del nodo move_group: entradas, URDF/SRDF del robot, joint states, TF, y salidas, trayectorias articulares hacia los controladores; el usuario habla con move_group por acciones/servicios o por la API MoveGroupInterface (moveit.picknik.ai, Concepts: Move Group).
- Planning scene: la representación interna del mundo, estado del robot + objetos de colisión + octomap de sensores, mantenida por el planning scene monitor; es contra ella contra lo que se comprueban colisiones (moveit.picknik.ai, Concepts: Planning Scene).
- SRDF en una frase: lo que el URDF no dice, grupos de planificación (el «brazo», la «pinza»), poses con nombre y pares de eslabones cuya colisión se ignora (moveit.picknik.ai, documentación del MoveIt Setup Assistant).

35–55 min OMPL: los planificadores de muestreo en producción

- Presentar OMPL (Open Motion Planning Library) como la biblioteca de planificadores de muestreo que MoveIt 2 usa por defecto vía plugin: contiene PRM, RRT, RRT-Connect (el defecto habitual), RRT* y decenas más (moveit.picknik.ai, Concepts: Motion Planning y OMPL Planner).
- Reconectar con S35: son los algoritmos de Lynch y Park (2017, pp. 379-385) operando en el C-space articular de 6 dimensiones; RRT-Connect es el RRT bidireccional (dos árboles que se conectan) de Lynch y Park (2017, p. 382).
- Recordar las propiedades para ajustar expectativas: probabilísticamente completos y no óptimos, de ahí caminos distintos en cada ejecución y la etapa posterior de suavizado/parametrización temporal antes de ejecutar (enlazar con las trayectorias del bloque 4).
- Demo en vivo con el panel Motion Planning de RViz (demo del Panda o del UR del curso): mover el efector objetivo, Plan, Execute; repetir el mismo objetivo dos veces para ver dos caminos distintos; añadir una caja como objeto de colisión a la planning scene y ver el desvío.

55–65 min Descanso (10 min)

- Pausa.

65–80 min Planificación cartesiana

- Distinguir planificación en espacio articular (objetivo = configuración; el camino del efector es «lo que salga») de la planificación cartesiana (el efector debe seguir una trayectoria en el espacio de tareas: soldar, dispensar, insertar).
- Explicar el método del camino cartesiano en MoveIt 2: interpolar la pose del efector por waypoints y resolver IK paso a paso (computeCartesianPath), con fracción alcanzada como resultado

(moveit.picknik.ai, tutoriales de la MoveGroupInterface); avisar de sus fallos típicos cerca de singularidades, conexión directa con el jacobiano y las singularidades del bloque 4.

- Criterio de elección para el proyecto: pick and place = articular con OMPL; gestos del efector con restricción de camino = cartesiano; y en la frontera, planificadores con restricciones.

80-105 min Taller de integración: el stack completo y el proyecto

- Componer en la pizarra, entre todos, el diagrama del stack completo del curso: sensores → percepción/SLAM (B6) → mapa → AMCL → Nav2 (S36) → /cmd_vel → base; y en paralelo cámara → planning scene → MoveIt 2 → brazo; todo sobre ROS 2/DDS (S32-S33), con TF2 como columna vertebral geométrica (S34) y Gazebo como banco de pruebas.
- Sesión de trabajo por equipos de proyecto (15 min): cada equipo sitúa su proyecto sobre el diagrama, identifica qué bloques usa, qué plugins/parámetros debe justificar y qué interfaz (topic/servicio/acción) expone cada módulo propio; el profesor rota por los equipos.
- Puesta en común relámpago: un riesgo técnico por equipo y su plan de mitigación (p. ej., «QoS incompatibles entre driver y Nav2», «TF sin sincronizar en la cámara», «OMPL no encuentra camino con la pinza cargada»).

105-110 min Síntesis del bloque

- Recorrer en 5 minutos las ideas fuerza: grafo de computación desacoplado; QoS como contrato; sim/real intercambiables; planificar = buscar en C-space con las propiedades de Lynch y Park (2017, p. 356) como brújula; Nav2 y MoveIt 2 como institucionalización de esos algoritmos.

110-120 min Cuestionario B7 (10 min)

- Cuestionario de seguimiento B7 (8-10 preguntas): QoS y grafo (docs ROS 2), topic/servicio/acción, comandos de colcon, admisibilidad de la heurística de A* (Corke, 2023, p. 175), PRM vs RRT y sus propiedades (Lynch y Park, 2017, pp. 378-385), capas del costmap y papel de AMCL (docs Nav2), planning scene y OMPL (docs MoveIt 2).

37.1. MoveIt 2: move_group y la planning scene

MoveIt 2 es la plataforma de manipulación del ecosistema ROS 2: integra planificación de movimiento, comprobación de colisiones, cinemática directa e inversa, ejecución de trayectorias y percepción 3D del entorno para robots manipuladores (moveit.picknik.ai, sección Concepts). Su pieza central es el nodo `move_group`, un integrador que reúne todas las capacidades y las expone mediante acciones y servicios de ROS 2, consumibles desde C++ (MoveGroupInterface), Python o el panel Motion Planning de RViz; como entradas consume la descripción cinemática del robot en URDF, su complemento semántico SRDF, grupos de planificación como «brazo» o «pinza», poses con nombre, exclusiones de colisión, los joint states publicados por el robot y el árbol TF2 de S34 (moveit.picknik.ai, Concepts: Move Group). El estado del mundo vive en la planning scene: la representación combinada del estado del robot y del entorno, objetos de colisión declarados por el usuario y un octomap construido desde sensores de profundidad, que mantiene actualizada el planning scene monitor; toda comprobación de colisión durante la planificación se hace contra ella (moveit.picknik.ai, Concepts: Planning Scene). El concepto merece subrayarse porque generaliza el costmap de ayer: donde Nav2 resume el mundo en una rejilla 2D de costes, MoveIt 2 lo representa en 3D con geometría exacta más voxels, porque un brazo colisiona con el codo, no solo con el efector.

37.2. OMPL: los planificadores de muestreo en producción

La planificación de MoveIt 2 es modular mediante plugins, y el plugin por defecto es OMPL, la Open Motion Planning Library: una biblioteca de código abierto que «consiste enteramente en planificadores de muestreo» y que MoveIt 2 configura y ejecuta contra la planning scene

(moveit.picknik.ai, Concepts: Motion Planning y página del OMPL Planner). Aquí la teoría de S35 se vuelve producto: el C-space es el espacio articular de 6 o 7 dimensiones, exactamente el régimen donde los métodos de rejilla son imprácticos (Lynch y Park, 2017, p. 378), el muestreador genera configuraciones articulares, el comprobador de colisiones las valida contra la planning scene y el planificador construye el árbol o el roadmap. El planificador por defecto habitual, RRT-Connect, es el RRT bidireccional de la sesión S35: dos árboles, uno desde la configuración inicial y otro desde la meta, que crecen alternándose e intentan conectarse (Lynch y Park, 2017, p. 382), una estrategia particularmente eficaz en espacios articulares. Las propiedades teóricas explican los comportamientos que los estudiantes verán en la demo: al ser probabilísticamente completos y no óptimos (Lynch y Park, 2017, pp. 356 y 383), dos planificaciones al mismo objetivo producen caminos distintos, a veces visiblemente subóptimos, y por eso el pipeline añade tras el planificador etapas de suavizado y de parametrización temporal, convertir el camino geométrico en trayectoria ejecutable con límites de velocidad y aceleración, enlazando con la generación de trayectorias del bloque 4.

37.3. Planificación cartesiana y criterio de elección

La planificación articular con OMPL resuelve «lleva el efector a esta pose» sin comprometerse con el camino intermedio del efector, y eso basta para el pick and place. Pero una familia de tareas industriales, soldadura, dispensado de adhesivo, pulido, inserción guiada, exige que el efector siga un camino cartesiano definido en el espacio de tareas. Para ello MoveIt 2 ofrece el cálculo de caminos cartesianos: la API `computeCartesianPath` interpola la pose del efector a lo largo de una secuencia de waypoints con un paso máximo y resuelve la cinemática inversa en cada paso, devolviendo además la fracción del camino que consiguió resolver (moveit.picknik.ai, tutoriales de la MoveGroupInterface). Esa fracción menor que uno es pedagógicamente valiosa: el camino cartesiano fracasa exactamente donde el bloque 4 predijo, cerca de singularidades el jacobiano pierde rango y la IK exige velocidades articulares enormes, y en los límites articulares o del espacio de trabajo la solución deja de existir. El criterio de elección que se lleva el estudiante al proyecto: objetivo de pose sin restricción de camino → planificación articular con OMPL; camino del efector restringido → planificación cartesiana (o planificadores con restricciones explícitas); y siempre, validación en la planning scene con la geometría real de la herramienta y la pieza.

37.4. El stack completo: síntesis del bloque y del proyecto

La última media hora del bloque se dedica a ensamblar el mapa mental completo, porque es el andamiaje del proyecto integrador. Sobre ROS 2 y DDS (S32-S33) como sistema nervioso, con TF2 (S34) como columna vertebral geométrica y Gazebo (S34) como banco de pruebas, el robot móvil encadena: sensores → mapeo/SLAM (bloque 6) → mapa estático → AMCL publicando map → odom → Nav2 con sus costmaps por capas, su planificador global de la familia Dijkstra/A* y su controlador local reactivo (S35-S36) → `/cmd_vel` → base. En paralelo, el manipulador encadena cámara → planning scene → OMPL en el C-space articular → trayectoria suavizada y temporizada → controladores (S37, con la cinemática del bloque 4 y el control del bloque 5 por debajo). Cada flecha del diagrama es una interfaz ROS 2 con su tipo y su QoS; cada caja es un conjunto de nodos con parámetros que hay que justificar, no solo copiar. El cuestionario B7 de los últimos diez minutos evalúa exactamente ese nivel: no la sintaxis de los comandos, sino saber qué pieza hace qué, con qué algoritmo, con qué propiedades y por qué está donde está.

Guía de conducción de las discusiones y puestas en común de este bloque

El método general de conducción está desarrollado en los apuntes del bloque 1 y aquí se aplica sin cambios: los diez segundos de silencio después de formular la pregunta, sin reformular y sin contestarse; la distinción entre la pregunta cerrada de recuperación, las de los tramos que abren S33, S35 y S36, y el cuestionario relámpago que cierra S33, que no se discuten sino que se corrigen en segundos porque su función es forzar recuperación activa, y la pregunta abierta de discusión, que se lanza al grupo entero y cuyo valor está en el razonamiento; la votación a mano alzada cuando conviene hacer visible un criterio; el reparto de la palabra; y la exploración de la respuesta errónea antes de corregirla nombrando fuente y sección. Lo que cambia es la materia. En este bloque las discusiones son de criterio técnico y nacen de resultados del taller, igual que en el bloque 6, pero con dos particularidades propias. La primera es que aquí el resultado del que nacen suele ser un fallo y no un número: los topics están y no llega nada, el robot tiembla en RViz, el planificador no encuentra camino con la pinza cargada. La segunda es que gran parte del material se cita sin página, porque para ROS 2, Gazebo, Nav2 y MoveIt 2 no existe edición paginada estable y se cita por nombre de documento y sección (docs.ros.org, docs.nav2.org, moveit.picknik.ai); eso cambia la mecánica del cierre, porque el gesto de zanjar señalando la página se sustituye por el de abrir la página de configuración en el proyector, que funciona igual de bien y además enseña a leer documentación.

De ahí tres consecuencias prácticas. La primera es que en este bloque el árbitro es la terminal: casi cualquier disputa sobre qué hace el sistema se resuelve ejecutando `ros2 topic hz`, `ros2 topic info` o `view_frames` delante de todos, y conviene tener las terminales abiertas durante las discusiones y no sólo durante el taller. La segunda es que los dos cuadernos del bloque están contruidos precisamente para producir el material de estas puestas en común: `82514_S34_Entorno_ROS2_TF2.ipynb` contiene la celda de diagnóstico que cada estudiante debe pasar antes del taller y la reimplementación en numpy del árbol de transformadas, con el experimento de la deriva en odom frente a la sierra acotada en map; y `82514_S35_Planificacion.ipynb` produce las cifras con las que se construye la tabla comparativa de planificadores, las longitudes de camino, las tasas de éxito y la dispersión entre semillas, sin las cuales esa tabla es una lista de adjetivos. La tercera es que las discusiones más valiosas del bloque no son las de teoría sino las dos de taller: la puesta en común de errores frecuentes de S34, que es la parte más rentable de esa sesión, y la composición del stack completo de S37, que es el andamiaje del proyecto integrador.

La regla añadida en este bloque

Ninguna afirmación sobre lo que hace el sistema se acepta sin comprobarla en la terminal. Si alguien dice que el nodo publica, se pide `ros2 topic hz`; si dice que la transformada existe, se pide `tf2_echo`; si dice que el planificador falla, se pide el mensaje de error literal. No es desconfianza: es que en un sistema distribuido la intuición sobre lo que está pasando es sistemáticamente peor que la introspección, y aprender ese reflejo es la mitad del bloque.

El ejercicio de QoS a mano alzada en S32 (50–55 min)

Objetivo

El apartado 32.3 ya da las respuestas: un flujo de sensor de alta frecuencia como `/scan` se publica best effort porque si se pierde un barrido el siguiente llega en veinticinco milisegundos y retransmitir sólo añadiría latencia, y un mapa que se publica una vez debe ser reliable y transient local para que un visualizador arrancado más tarde lo reciba, combinación que ningún perfil predefinido ofrece, porque «sensor data», «services» y el perfil por defecto son todos volatile y la durabilidad hay que declararla a

mano (docs.ros.org, About Quality of Service settings). Si la discusión se conduce como una comprobación de esas tres respuestas, sobra. Lo que aporta y la exposición no puede aportar es que el estudiante descubra que la elección de QoS no es una preferencia sino una consecuencia de dos preguntas sobre el canal: qué pasa si se pierde un mensaje, y a quién le hace falta el último mensaje si llega tarde. Con esas dos preguntas se resuelven canales que no están en ninguna lista, y eso es lo que se necesita en diciembre. El segundo objetivo es preventivo y muy concreto: dejar clavada la regla de compatibilidad antes del taller, porque una pareja reliable contra best effort no da error, simplemente no fluye nada, y ese fallo silencioso es el que más horas consume en los proyectos.

Formato y tiempos

El tramo de 40 a 55 minutos es el de DDS y QoS, y el ejercicio es su último punto: cinco minutos, del 50 al 55, con el cierre de la sesión de 55 a 60. El reparto: en el minuto 50 se escriben en la pizarra los tres topics en columna y, al lado, las dos preguntas de decisión; se enuncia el ejercicio y se callan diez segundos. De 50 a 53, las tres votaciones a mano alzada, una por topic, con recuento escrito y sin comentar. De 53 a 55, las razones y la regla de compatibilidad. La mecánica de la votación conviene respetarla al detalle porque es lo que hace hablar a un grupo que lleva veinte minutos oyendo hablar de middleware: se enuncia la opción, se pide que voten todos a la vez a una señal, se cuenta en voz alta y se escribe el número sin valorarlo. Y hay una decisión de formato que decide el rendimiento: se vota primero y se razona después, no al contrario, porque quien ya ha comprometido su mano tiene algo que defender.

Pregunta de apertura

Literalmente, con los tres topics escritos: «Tres canales de nuestro robot: /scan, que publica el láser a cuarenta hercios; /cmd_vel, que lleva las consignas de velocidad a los motores; y /map, que se publica una sola vez al arrancar. Para cada uno voy a pedirlos dos manos: fiable o mejor esfuerzo, y si hace falta que un suscriptor que llegue tarde reciba el último mensaje. No quiero justificaciones todavía, quiero manos.» Y después de cada votación, la pregunta que produce el contenido: «¿Qué pasa exactamente si se pierde un mensaje de este canal?» Formular el ejercicio como dos decisiones binarias y no como qué QoS pondríais es lo que lo hace ejecutable en cinco minutos: qué QoS pondríais admite un discurso, y esto admite una mano.

Respuestas y resultados que van a salir, y qué hacer con cada uno

/scan: mayoría abrumadora por fiable. Es el resultado que sale casi siempre, típicamente veinticinco manos contra cinco, y es la equivocación productiva del ejercicio: el razonamiento subyacente es que los datos del sensor son importantes, luego hay que garantizarlos. No se corrige de frente; se pregunta qué hace el sistema mientras espera la retransmisión de un barrido perdido. La respuesta, esperar un dato viejo teniendo uno nuevo disponible, la construye la clase sola, y de ahí sale la formulación correcta: en un flujo periódico y redundante, la fiabilidad se paga en latencia y la latencia es peor que la pérdida. Conviene rematar con el número: a cuarenta hercios el siguiente barrido llega en veinticinco milisegundos, y ningún mecanismo de retransmisión es más rápido que eso. Es el mismo argumento que en S27 del bloque 6 hacía preferir un detector rápido a uno preciso.

/cmd_vel: la clase se divide. Es la votación más interesante y hay que trabajarla, porque las dos posiciones tienen argumento. Quien vota fiable dice que una consigna perdida es una consigna que el robot no ejecuta; quien vota mejor esfuerzo dice que el controlador publica a diez o veinte hercios y que el siguiente comando corrige. Los dos tienen razón en un caso distinto, y la pregunta que lo aclara es qué significa el mensaje: si /cmd_vel lleva una consigna que se mantiene hasta la siguiente, perder un mensaje es perder actualidad y el flujo se autorrepara; si el diseño fuera de comandos con efecto acumulativo, perder uno sería perder estado y habría que garantizarlo. La lección de ingeniería, que vale para todo el bloque, es que la QoS de un canal se decide leyendo la semántica del mensaje, no el

nombre del topic. Y el remate práctico: además de la QoS, este canal necesita un vigilante de plazo, porque lo peligroso no es perder un mensaje sino dejar de recibirlos con el robot en marcha, que es exactamente para lo que existe la política de deadline (docs.ros.org, About Quality of Service settings).

/map: nadie ve la durabilidad hasta que se les pregunta. En la primera votación casi todos aciertan con fiable y casi nadie pide que el suscriptor tardío reciba el mensaje, porque es la política menos intuitiva de las cuatro. La forma de hacerla evidente es un escenario en dos frases: el mapa se publica una vez, en el segundo tres del arranque; abris RViz en el segundo cuarenta. ¿Veis el mapa? La respuesta es no, y el gesto de incomodidad que produce es el mejor argumento posible para transient local. Conviene añadir que este caso concreto va a ocurrirles en el taller de S36 y que cuando ocurra sabrán qué mirar, lo cual convierte cinco minutos de teoría en una hora de laboratorio ahorrada.

«¿Y si me equivoco de QoS, qué pasa?». Es la mejor pregunta del tramo y si no sale hay que provocarla, porque la respuesta es la regla que hay que dejar escrita en la pizarra: un publicador y un suscriptor sólo se conectan si sus políticas son compatibles, y una pareja incompatible no produce ningún error, simplemente no fluye ningún mensaje. Hay que decirlo con las palabras que van a usar ellos, los topics están, ros2 topic list los enseña, y ros2 topic echo no imprime nada, y dar de inmediato el procedimiento de diagnóstico: ros2 topic info con el detalle de perfiles, que muestra los publicadores y suscriptores y sus políticas. Esa secuencia de tres comandos es el contenido más rentable de la sesión entera.

El error de razonamiento más frecuente y cómo desmontarlo

El error dominante es identificar fiabilidad con importancia: cuanto más importante es el dato, más fiable debe ser el canal. Es un error de categoría y sobrevive a cualquier explicación teórica, porque suena a sentido común. La única forma de desmontarlo es con el experimento, y por suerte cuesta noventa segundos en la clase siguiente y conviene anunciarlo aquí: en el taller de S34, con el TurtleBot simulado, se mide ros2 topic hz /scan con el perfil de datos de sensor y con perfil fiable en una red cargada, y se compara el retardo entre el sello de tiempo del mensaje y el instante de recepción. El resultado, la versión fiable entrega todos los barridos y algunos con retardo acumulado, convierte una discusión de opiniones en una medida. Mientras llega ese momento, el argumento que funciona en el aula es el de la analogía con el bloque 3: nadie retransmite una lectura de un termopar, se toma la siguiente, y nadie duda de que la temperatura sea importante. El segundo error, más específico y muy caro en el proyecto, es creer que la QoS se configura una vez en el sistema; se desmonta señalando que cada publicador y cada suscriptor la declara por separado, y que por tanto la compatibilidad es una propiedad de cada pareja y hay que verificarla pareja a pareja.

Si la clase no habla

Con votación a mano alzada el silencio es casi imposible, y si aparece es porque la clase no entiende las opciones, no porque no quiera pronunciarse. La maniobra de rescate es traducir las políticas a lenguaje llano antes de votar, escribiendo en la pizarra las cuatro decisiones en forma de preguntas: si se pierde, ¿lo reintento o paso al siguiente? Si alguien llega tarde, ¿le doy el último o que espere? ¿Cuántos guardo en la cola? ¿Cuánto tiempo tolero sin recibir? Con esas cuatro preguntas cualquiera vota. La segunda maniobra, si el grupo sigue mudo, es dar un cuarto canal más fácil como calentamiento, la temperatura de la batería, que se publica cada diez segundos, porque los casos fáciles desbloquean a quien tiene miedo de equivocarse en el difícil. Y una tercera para grupos muy pasivos: el profesor declara en voz alta una configuración deliberadamente absurda, el mapa en best effort con cola de uno, y pide que digan qué va a fallar; refutar es siempre más barato que proponer.

Cierre

La idea que debe quedar, dicha en una frase y escrita: la QoS es un contrato entre dos extremos que se deduce de la semántica del canal, no de la importancia del dato, y las dos preguntas que lo resuelven son qué cuesta perder un mensaje y quién necesita el último si llega tarde. El corolario operativo, que conviene dictar porque se cobra en el taller: cuando un topic exista y no llegue nada, la primera sospecha es la incompatibilidad de QoS y el primer comando es `ros2 topic info` con detalle. Lo que no conviene decir es que en la práctica se dejan los valores por defecto y ya funciona: es verdad a medias, los perfiles predefinidos están bien elegidos, y es exactamente la frase que garantiza tres horas perdidas el día que se mezcle un driver de fabricante con un nodo de Nav2. Tampoco conviene entrar en la comparación de proveedores de DDS ni en la configuración de descubrimiento, aunque alguien lo pregunte: se nombra que la capa `rmw` permite intercambiarlos sin tocar el código de usuario (docs.ros.org, About different ROS 2 DDS/RMW vendors), se anota la pregunta y se sigue, porque el cierre de la sesión tiene que dejar dicho el deber previo al taller.

La pregunta sembrada de S34 (103–108 min): quién publicará `map` a `odom`

Objetivo

El guion la formula como pregunta de conexión para dejar sembrada, y esa etiqueta describe exactamente su función: no es una discusión que deba resolverse hoy, es una incomodidad que debe quedar instalada hasta S36. El apartado 34.3 ya da la respuesta, la convención de REP 105 encadena `map` a `odom` y `odom` a `base_link`, la odometría publica la segunda arista y el sistema de localización la primera, precisamente para corregir la deriva, pero leerlo produce la sensación de haber entendido una convención administrativa. El objetivo de estos cinco minutos es que la clase deduzca por qué hacen falta dos aristas y no una, es decir, que descubra que ningún sistema de referencia puede ser a la vez exacto y continuo, y que esa imposibilidad es la razón de la arquitectura. Ése es el aprendizaje: no la lista de frames, sino que `map` es exacta y discontinua y `odom` es continua e inexacta, y que un controlador reactivo necesita continuidad mientras un planificador necesita exactitud. Con eso, en S36 la separación entre `costmap` global anclado a `map` y `costmap` local anclado a `odom` no habrá que explicarla.

Formato y tiempos

La parte 3 del taller ocupa de 85 a 110 minutos y esta pregunta va al final, en los cinco minutos que van del 103 al 108, con el árbol de transformadas del TurtleBot ya generado y proyectado en pantalla, sin el `frames.pdf` delante la discusión no tiene objeto. El reparto: en el minuto 103 se señala en la pantalla la cadena `odom` a `base_footprint` a `base_link` a `base_scan`, se pregunta quién publica cada arista y se deja que lo contesten, que es recuperación pura y cuesta un minuto. De 104 a 107 se enuncia la pregunta sembrada y se discute. De 107 a 108 se cierra sin dar la respuesta completa. Un aviso de conducción que es todo el arte de este tramo: la tentación de explicar aquí la localización de Nav2 es muy fuerte y hay que resistirla, porque quema el material de S36 y porque a los ciento ocho minutos de taller la clase no puede absorber una arquitectura nueva. Lo que se busca es dejar una pregunta abierta, no cerrarla.

Pregunta de apertura

Literalmente, señalando la raíz del árbol en la pantalla: «Mirad quién es la raíz de vuestro árbol: `odom`. No hay ningún frame `map`. Ahora decidme dos cosas: si le pido a este robot que vaya al muelle de carga, ¿en qué frame doy esa coordenada? Y si la odometría lleva media hora acumulando error, ¿qué le pasa a mi orden?» La formulación pide un caso de uso y no una definición, que es lo que la hace productiva. Y hay una segunda pregunta que conviene reservar para el minuto 106, cuando ya haya

quien proponga añadir un frame map: «¿Y por qué no arreglamos directamente odom a base_link, corrigiéndola cada vez que sabemos dónde estamos de verdad?» Ésta es la pregunta que produce el aprendizaje, porque la respuesta es la única que no es evidente.

Respuestas y resultados que van a salir, y qué hacer con cada uno

«**La orden la doy en odom, pero se irá desviando**». Es la respuesta correcta y llega en seguida porque acaban de ver la deriva con tf2_echo. Se acepta y se cuantifica, que es lo que la convierte en contenido: con la odometría del ejercicio 2 del cuaderno 82514_S34_Entorno_ROS2_TF2.ipynb acumulando tres décimas de grado por paso, el error crece de forma monótona y sin ninguna cota, y a los pocos minutos de recorrido la coordenada del muelle expresada en odom ya no señala el muelle. Conviene pedir que digan qué gráfica tenía ese error, creciente y sin techo, porque la comparación con la del otro caso es todo el argumento del tramo.

«**Pues hay que añadir un frame que sea el mapa**». Es la propuesta que se busca y hay que acogerla y complicarla de inmediato con la segunda pregunta: por qué no corregir directamente la arista de la odometría. La respuesta que hay que arrancar es que una corrección de localización llega a saltos, y que si esos saltos se meten en odom a base_link, todo lo que consume esa transformada ve teletransportarse el mundo. En el ejercicio del cuaderno esto se ve con una imagen que conviene nombrar: en map el error es una sierra acotada, que vuelve a cero cada vez que se corrige, y en odom es una rampa sin cota. Ninguno de los dos comportamientos es el bueno para todo, y de ahí que REP 105 obligue a tener las dos aristas.

«**¿Y quién publica esa arista?**». Es la pregunta que cierra el tramo y hay que contestarla sólo a medias, deliberadamente: el sistema de localización, que es un filtro de los que estudiaron en el bloque 6 corriendo sobre el mapa que ellos mismos guardaron, y que veremos con nombre y apellidos en S36. Conviene añadir la frase que hace que la promesa se recuerde: cuando en dos clases arranquemos Nav2 y haya que fijar la pose inicial con el ratón en RViz, lo que estaremos haciendo es darle la primera estimación a ese filtro para que pueda empezar a publicar esta arista. Y ahí se corta.

«**Entonces la pose del robot no la publica nadie**». Es la observación más fina que puede salir y merece medio minuto porque desmonta un malentendido muy extendido: en efecto, el sistema de localización no publica la pose del robot, publica una corrección entre dos sistemas de referencia, y la pose se obtiene componiendo la cadena. La consecuencia práctica hay que decirla porque ahorra depuraciones absurdas: si alguien busca un topic con la pose y no lo encuentra, no es que falte, es que la pose es el resultado de una consulta al árbol. Y la consecuencia de diseño: quien publica una arista es responsable de una relación, no de un dato, que es la diferencia entre un sistema componible y un montón de nodos.

El error de razonamiento más frecuente y cómo desmontarlo

El error dominante en este punto del taller es tratar el árbol de transformadas como una lista de nombres que hay que respetar por convención, sin ver que cada arista tiene un dueño y una física distinta. El síntoma aparece en el proyecto en forma de dos nodos publicando la misma arista, que es el error que el apartado 34.3 anticipa: el árbol es árbol y no grafo, cada frame tiene un único padre, y dos fuentes sobre la misma arista producen los saltos erráticos que los estudiantes describen como que el robot tiembla en RViz. Se desmonta con datos del propio taller y no con una advertencia: basta arrancar un segundo publicador de odometría sobre odom a base_link en una terminal y mirar RViz durante quince segundos. El temblor es tan evidente y tan ridículo que la lección se queda. Y conviene aprovechar para dar el diagnóstico, que es lo que de verdad necesitan: view_frames muestra quién publica cada arista y a qué frecuencia, y una arista con dos emisores se detecta ahí en cinco segundos. El segundo error, más conceptual, es esperar que TF2 devuelva una transformada para cualquier instante; se desmonta con la solución del ejercicio 3 del cuaderno, que explica por qué el búfer

interpola entre dos muestras y prefiere fallar antes que extrapolar, extrapolar es inventarse datos, y esos datos se van a usar para decidir dónde está un obstáculo.

Si la clase no habla

A los ciento tres minutos de un taller de dos horas la clase está cansada y con la cabeza en su propia pantalla, de modo que hay que bajar el coste de participar al mínimo. La maniobra de rescate es señalar y preguntar por nombre sobre la pantalla: quién publica esta arista, y esta, y esta. Son tres preguntas de una palabra y reactivan al grupo. La segunda maniobra es la votación binaria con el dibujo delante: quién dice que la corrección de localización debe modificar la arista de la odometría y quién dice que hace falta una arista nueva; con el recuento escrito se pide una razón a cada bando. La tercera, la más eficaz cuando el grupo está agotado, es hacerlo con el cuerpo: el profesor camina en línea recta por el aula contando pasos en voz alta y anunciando su posición estimada, y a mitad del recorrido alguien de la clase le dice dónde está de verdad; el salto que el profesor tiene que dar para reconciliar las dos cosas es literalmente la arista map a odom, y verlo hecho con los pies lo entiende todo el mundo.

Cierre

La idea que debe quedar, y conviene dejarla escrita en tres líneas en la pizarra, es la tabla de REP 105 con su física: odom a base_link es continua y con deriva, la publica la odometría; map a odom es exacta y con saltos, la publica la localización; base_link a base_scan es fija y calibrada, la publica la descripción del robot. Y el corolario, que es el que se cobra dos clases más tarde: ningún frame es a la vez exacto y continuo, y por eso hacen falta dos. El cierre correcto de este tramo es una promesa y no una respuesta: la arista que falta la publicará en S36 el filtro de partículas del bloque 6, y esa frase basta. Lo que no conviene hacer es explicar aquí la configuración de la localización de Nav2: se queda sin efecto porque falta el contexto y se pierde el efecto de S36. Tampoco conviene decir que estas convenciones son cosa de ROS y que en otro middleware serían otras; es cierto y es irrelevante, y además desactiva la única cosa que hace valiosa una convención, que es que todo el mundo la respete.

La puesta en común de errores frecuentes del taller de S34 (110–117 min): la parte más valiosa de la sesión

Objetivo

El guion dedica el tramo final a recoger evidencias del taller, y ahí es donde hay que insertar la puesta en común de errores, que es la actividad de mayor rendimiento por minuto de todo el bloque. La razón es sencilla: en un taller de ROS 2 cada pareja tropieza tres o cuatro veces, resuelve sus tropiezos en privado y se lleva a casa la impresión de que a los demás les ha ido mejor. Poner los tropiezos en común hace tres cosas que la teoría no puede hacer. Primero, convierte una anécdota individual en un patrón de sistema: quince parejas con el mismo error significa que el error no es de la pareja, es del modelo mental que la clase tiene del entorno. Segundo, construye un repertorio de diagnóstico, qué síntoma corresponde a qué causa y qué comando lo distingue, que es exactamente la competencia que se evalúa en el proyecto y que ningún apunte transmite. Y tercero, y no es menor, desactiva el sentimiento de incompetencia particular, que en un máster con estudiantes de formación no informática es un problema real y silencioso. La diferencia con la teoría es total: los apuntes ya dicen que hay que hacer source del overlay y que el árbol no admite dos padres, y lo dicen desde S33; la puesta en común es lo que convierte esas frases en reflejos.

Formato y tiempos

El tramo de 110 a 120 minutos incluye la recogida de evidencias y el anuncio de S35, de modo que la puesta en común tiene siete minutos, del 110 al 117. El reparto que funciona, y es un formato que conviene mantener idéntico todo el curso porque su valor está en la repetición: de 110 a 112, dos minutos de recogida rápida, pidiendo a cada pareja un solo error y en una sola frase, con el profesor escribiendo en la pizarra tres columnas, síntoma, causa, comando que lo distingue, y sin comentar ninguno todavía. De 112 a 116, cuatro minutos para completar la tabla con la clase: para cada síntoma recogido se pregunta a otra pareja qué comando lo diagnostica. De 116 a 117, el cierre. Dos avisos que deciden si esto funciona. El primero es exigir el síntoma antes que la causa: si se pregunta qué error habéis tenido, la respuesta es una explicación ya interpretada y casi siempre equivocada; si se pregunta qué visteis en pantalla, la respuesta es un dato. El segundo es que el profesor no debe resolver: se limita a escribir y a redirigir la pregunta a otra pareja, porque siete minutos de profesor explicando errores no sirven de nada y siete minutos de estudiantes explicándose entre ellos son la mejor clase de la semana.

Pregunta de apertura

Literalmente, con las tres columnas ya dibujadas: «Antes de recoger nada quiero un dato de cada pareja. No me contéis qué creáis que pasaba: decidme qué visteis en la pantalla, la primera vez que algo no funcionó, y con qué comando descubristeis por qué. Empezamos por aquí y damos la vuelta al aula.» La formulación tiene tres decisiones deliberadas. Pedir lo que se vio y no lo que se cree separa el síntoma del diagnóstico, que es la distinción que se quiere enseñar. Pedir la primera vez evita que cada pareja cuente su historia entera. Y anunciar que se da la vuelta al aula garantiza que hablen todos y no los cuatro de siempre, con el efecto añadido de que quien sabe que le va a tocar prepara la respuesta.

Errores que van a salir y qué hacer con cada uno

«**ros2 run no encontraba el paquete**». Es el error más repetido, con diferencia, y su causa es siempre la misma familia: la terminal no tiene el overlay en su entorno, o lo tiene de un install obsoleto. Aquí lo importante no es dar la solución, que ya está en el apartado 33.3, sino escribir en la columna del comando lo que distingue una causa de la otra: `ros2 pkg prefix` con el nombre del paquete dice si el entorno lo ve y desde dónde, y eso separa en dos segundos el olvido del source de la construcción fallida. Conviene pedir a mano alzada cuántas parejas lo han tenido; el bosque de manos es el argumento entero, porque convierte un descuido personal en una propiedad del modelo underlay/overlay que todos tienen que interiorizar.

«**El nodo arrancaba pero no llegaba nada al otro lado**». El segundo más frecuente, y el más instructivo porque tiene tres causas posibles con el mismo síntoma: nombre de topic distinto, muy a menudo por un remapeo o por una barra inicial olvidada, tipo de mensaje distinto, o políticas de QoS incompatibles, que es el fallo silencioso que se anunció en S32. La columna del comando es la que hay que llenar con cuidado, y en este orden: `ros2 topic list` con tipos para descartar el nombre y el tipo, y `ros2 topic info` con detalle para ver publicadores, suscriptores y perfiles. Es la ocasión de cobrar la discusión de QoS de la primera sesión del bloque, y hay que decirlo en voz alta: esto es lo que os avisé el miércoles y ha pasado exactamente así.

«**Nos faltaba un paquete que no venía instalado**». Sale cada año y siempre es el mismo: el teleoperador de teclado no viene en el metapquete de escritorio y hay que instalarlo aparte, tal como recoge la solución del ejercicio 1 del cuaderno 82514_S34_Entorno_ROS2_TF2.ipynb. Conviene tratarlo sin dramatismo y extraer la regla, que es de método y no de ROS: la comprobación previa útil no es contar paquetes, una instalación de escritorio ronda los trescientos o cuatrocientos, sino verificar la lista concreta de los que hace falta, que es lo que hace la celda de diagnóstico del cuaderno.

Y de ahí la consigna para el proyecto: la lista de dependencias se declara en package.xml y se verifica con la máquina en la que se va a demostrar, no con la del que programa.

«**El robot temblaba en RViz**» o «**TF daba error de extrapolación**». Es el error más avanzado de los que van a salir y el que más conviene celebrar, porque lo tiene la pareja que ha llegado más lejos. El temblor es doble publicación sobre la misma arista y se diagnostica con view_frames, que enseña emisores y frecuencias. El error de extrapolación es una consulta a un instante que el búfer no cubre, y hay que aprovecharlo para dejar dicho por qué TF2 prefiere fallar a extrapolar, que es la lección honesta de la sesión: extrapolar es inventarse una posición y esa posición se va a usar para decidir si hay un obstáculo delante. Si aparece este error, es el que conviene poner al final de la tabla y dejar como imagen de cierre.

El error de razonamiento más frecuente y cómo desmontarlo

El error de razonamiento que hay que atacar no es ninguno de los de la tabla: es el método de depuración por conjetura y reintento, que consiste en cambiar cosas hasta que funciona sin haber identificado la causa. Es el hábito más caro que traen y el más difícil de corregir, porque a veces funciona. Se reconoce por una frase que dirán textualmente en la puesta en común: no sabemos qué pasaba, hemos reconstruido y ha funcionado. Cuando aparezca, y aparecerá, hay que detenerse quince segundos y hacer la única pregunta que lo desmonta: si mañana vuelve a pasar, ¿sabéis qué mirar? La respuesta es no, y eso es lo que hay que hacer visible, sin reproche, porque es la definición operativa de no haberlo arreglado. Y el argumento no puede ser de autoridad: tiene que apoyarse en la tabla que la propia clase acaba de construir, donde hay tres síntomas idénticos con tres causas distintas y un comando que las separa. La conclusión hay que dictarla porque es transferible a todo el curso: en un sistema distribuido, la conjetura tiene un espacio de búsqueda enorme y la introspección lo reduce a uno, y el coste de aprenderse cuatro comandos es infinitamente menor que el de reconstruir a ciegas. El segundo error de razonamiento, más benigno, es suponer que el problema está en el propio código cuando está en el entorno; se desmonta con la observación estadística de la propia pizarra, donde la mayoría de los síntomas recogidos no eran de código.

Si la clase no habla

Aquí el silencio tiene una causa concreta y hay que nombrarla para desactivarla: nadie quiere ser el que confiese que perdió veinte minutos por no hacer source de un fichero. La maniobra de rescate es que el profesor empiece confesando el suyo, y conviene tener uno de verdad y contarlos con detalle, el año pasado, media hora buscando por qué no llegaba nada a un nodo que estaba escuchando el topic con la barra delante. Una confesión del profesor abre la puerta a todas las demás y cuesta cuarenta segundos. La segunda maniobra es despersonalizar por escrito: en lugar de preguntar en voz alta, se pide que cada pareja escriba su error en un papelito sin nombre, se recogen y el profesor los lee en voz alta y los agrupa en la pizarra. Es más lento pero funciona con cualquier grupo y produce además una tabla más honesta. La tercera, para cuando no hay tiempo ni para eso, es la votación de la lista: el profesor enuncia los cinco errores clásicos y pide manos en cada uno; el recuento se escribe y ya hay conversación, porque nadie se siente señalado cuando levantan la mano otros dieciocho.

Cierre

La idea que debe quedar no es la lista de errores, que se olvida, sino la estructura: en un sistema distribuido, síntoma y causa no se corresponden uno a uno, y la única forma de pasar de uno a otra es la introspección. Conviene decirlo con la frase del apartado 32.2, que aquí cae sola: un ingeniero que domina estas herramientas puede diagnosticar un robot ajeno sin leer una línea de su código. El corolario es la consigna del proyecto: la tabla de la pizarra se fotografía, se sube al campus con las evidencias del taller y se amplía cada vez que aparezca un síntoma nuevo, porque es el documento más útil que van a tener en diciembre. Lo que no conviene hacer es rematar diciendo que estos errores

son normales y no hay que preocuparse: la mitad es verdad, son normales, y la otra mitad desactiva la lección, porque lo que hay que aprender es que son diagnosticables. Tampoco conviene señalar a la pareja que más problemas ha tenido, ni siquiera con simpatía, porque el precio de esa broma es que en el siguiente taller nadie declara nada.

La tabla comparativa de planificadores en S35 (45–55 min) y su reutilización en la apertura de S36

Objetivo

El guion pide un ejercicio en parejas de siete minutos y rellenar entre todos una tabla comparativa con cinco planificadores y cinco preguntas. El apartado 35.1 ya ha dado el vocabulario de propiedades, completo, completo en resolución, probabilísticamente completo, óptimo o satisfactorio, multiconsulta o consulta única, complejidad (Lynch y Park, 2017, pp. 355-356), de modo que la discusión no está para introducirlo, sino para que ese vocabulario se convierta en criterio de elección. Lo que debe quedar aprendido son dos cosas. La primera es que las propiedades no son etiquetas académicas: cada una responde a una pregunta que un ingeniero se hace de verdad, y hay una traducción directa entre ambas que conviene explicitar, completitud es qué me garantizas, optimalidad es cuánto camino de más recorro, multiconsulta es dónde gasto el cómputo, dimensión es si mi robot cabe. La segunda, y es la que sólo se aprende discutiendo, es que la elección casi nunca se decide por la propiedad más prestigiosa sino por la restricción del proyecto: el mapa cambia, la meta cambia, el robot tiene seis grados de libertad o el ordenador tiene un presupuesto cerrado. La misma tabla se reutiliza cinco días después en el arranque de S36 con la pregunta del almacén con carretillas, y esa continuidad hay que anunciarla, porque una tabla que se usa dos veces se estudia y una que se usa una vez se copia.

Formato y tiempos

El guion asigna de 45 a 55 minutos a campos potenciales y al ejercicio de síntesis, con el cierre de 55 a 60. El reparto que funciona: de 45 a 48, la exposición de los campos potenciales con la figura de los mínimos locales (Lynch y Park, 2017, pp. 386-388), que es el quinto planificador de la tabla y hay que tenerlo antes de rellenarla. En el minuto 48 se dibuja la tabla vacía, cinco filas, cinco columnas, y se reparten las filas por zonas del aula, una por pareja o grupo de parejas. De 48 a 51, tres minutos en parejas para rellenar su fila y sólo su fila. De 51 a 55, cuatro minutos de barrido: cada grupo dicta su fila y el resto puede impugnarla. Dos decisiones de formato que son la diferencia entre una tabla viva y un dictado. La primera es repartir por filas y no pedir la tabla completa a todos: con siete minutos, una pareja no puede razonar veinticinco celdas, y el reparto convierte a cada grupo en responsable de algo. La segunda es habilitar explícitamente la impugnación, quien no esté de acuerdo con una celda lo dice y hay que resolverlo, porque el desacuerdo entre grupos es donde está el contenido y sin permiso para discrepar nadie discrepa.

Pregunta de apertura

Literalmente, con la tabla vacía dibujada: «Cada grupo tiene un planificador y cinco preguntas: ¿completo?, ¿óptimo?, ¿dónde está el coste, en planificar o en consultar?, ¿sirve para seis grados de libertad?, ¿sirve con un mapa que cambia? Rellenad sólo vuestra fila, y para cada celda quiero la palabra exacta del vocabulario, no un sí o un no.» Exigir la palabra exacta, completo en resolución, probabilísticamente completo, es lo que impide que la tabla se convierta en una fila de aspas, y además obliga a volver al apartado 35.1. Y para el barrido conviene una segunda pregunta, que es la que produce la conversación: «¿Alguien discute alguna celda de esta fila?»

Respuestas y resultados que van a salir, y qué hacer con cada uno

«**A* es completo y óptimo**». Aparece siempre así, sin matiz, y el matiz es todo: es completo en resolución y óptimo sobre la rejilla, y sólo devuelve el camino de menor coste si la heurística es admisible, es decir, si no sobreestima (Corke, 2023, p. 175). Hay que exigir las dos precisiones y sostenerlas con el resultado del taller, porque el cuaderno 82514_S35_Planificacion.ipynb lo tiene medido: Dijkstra expande prácticamente todo el espacio libre, A* con heurística euclídea encuentra el mismo camino de 12,66 metros explorando una fracción de los nodos, y A* con la heurística multiplicada por dos expande diez veces menos pero devuelve un camino claramente peor. La conclusión que hay que dejar escrita es la del cuaderno: la heurística no cambia la respuesta, cambia el trabajo, siempre que sea admisible. Y conviene añadir el matiz profesional, porque es el que usarán: el A* ponderado existe y se usa a sabiendas, cuando hay que replanificar muchas veces por segundo y un camino un veinte por ciento peor calculado diez veces más rápido es el buen negocio.

«**PRM es probabilísticamente completo**» y **nadie sabe qué significa en la práctica**. Es la celda donde el vocabulario se vuelve abstracto y hay que aterrizarlo con los números del taller, que son inmejorables para esto: con treinta muestras, el roadmap encuentra camino en cuatro de ocho intentos, y con ciento veinte muestras en ocho de ocho. Ésa es la definición operativa de probabilísticamente completo, y verla en una tabla de éxitos y fracasos es lo que la hace comprensible. La segunda celda que hay que trabajar en esta fila es la de optimalidad, con la misma munición: 15,7 metros con treinta muestras frente a 12,7 con quinientas, contra los 12,66 de A*. No es óptimo, y se acerca al óptimo pagando muestras. El modo de fallo hay que nombrarlo porque es el que verán en Movelt 2: los pasajes estrechos, poco probables de muestrear, son su talón de Aquiles, y la figura del libro que contrasta cincuenta vértices mal conectados con trescientos bien conectados es la imagen que hay que proyectar (Corke, 2023, p. 187).

«**RRT es peor que A***, entonces **para qué se usa**». Es la impugnación más frecuente en el barrido y hay que darle la razón antes de contestarla, porque en el plano tiene razón: en el mapa del taller el RRT devuelve caminos de entre 15,1 y 24,5 metros según la semilla, es decir, hasta el doble del óptimo, y esa dispersión entre semillas conviene decirla con esas palabras, el mismo algoritmo, el mismo mapa, respuestas muy distintas. La respuesta a para qué se usa es la columna que casi nadie ha rellenado bien: los métodos de rejilla son impracticables más allá de unos pocos grados de libertad (Lynch y Park, 2017, p. 378), y un brazo de seis ejes no cabe en ninguna rejilla. Con eso la tabla revela su estructura: la columna de la dimensión no es una celda más, es la que decide, y es la razón de que en S37 los planificadores de Movelt 2 sean todos de muestreo. Conviene además dar la contrapartida honesta que el propio cuaderno señala: el RRT acepta restricciones de movimiento en su planificador local, y por esa puerta entra la cinemática del vehículo, con primitivas de Dubins para el coche no holonómico (Corke, 2023, pp. 196-198).

«**Los campos potenciales no valen para nada, se quedan atrapados**». Es la conclusión precipitada de la fila más fácil de despachar, y hay que reconducirla porque el planificador reactivo es la mitad de la arquitectura que van a usar mañana. Sí, el robot puede quedar atrapado en un mínimo local aunque exista camino factible, y la figura del libro lo muestra con un mínimo global, un mínimo local y cuatro puntos de silla generados por tres obstáculos (Lynch y Park, 2017, pp. 386-388). Pero la columna del mapa cambiante es la suya: como el gradiente se evalúa en tiempo real, puede manejar obstáculos que se mueven o aparecen inesperadamente (Lynch y Park, 2017, p. 386), y ningún planificador global hace eso a la frecuencia necesaria. La síntesis que hay que arrancar de la clase es la arquitectura: potenciales o cualquier controlador reactivo por debajo, planificador global por encima, que es literalmente la división planner/controller de Nav2 que se ve mañana.

El error de razonamiento más frecuente y cómo desmontarlo

El error dominante es buscar en la tabla el mejor planificador, es decir, leer las columnas como puntuaciones y sumarlas. Es el mismo vicio que en el bloque 6 amenazaba a la tabla de decisión de percepción y se corrige con la misma disciplina: obligar a nombrar la restricción que manda y la propiedad que se sacrifica. Pero en este bloque hay además un error más específico y más caro, que consiste en creer que la elección del algoritmo es la decisión importante, cuando la decisión importante suele estar antes. La demostración está en el ejercicio 1 del cuaderno 82514_S35_Planificacion.ipynb y merece proyectarse en el minuto 54 aunque cueste un minuto del cierre: con un robot de quince o veintiocho centímetros de radio, el camino cruza por la puerta estrecha y mide unos 13,2 metros; con cuarenta centímetros, la puerta estrecha se cierra, queda sin ninguna celda libre para el centro del robot, y el planificador se ve obligado a dar la vuelta por la puerta ancha, con un camino de unos 14,5 metros y una ruta completamente distinta. Ningún cambio de planificador arregla eso, y ningún planificador es culpable de eso. La lección de ingeniería que hay que dictar es que el radio del robot no es un parámetro del software sino una decisión de diseño mecánico con consecuencias topológicas, y que doce centímetros de más en el chasis pueden eliminar una conexión entre naves de un almacén ya construido. Es también, dicho de paso, la mejor razón para hacer esta comprobación antes de comprar el vehículo.

Si la clase no habla

Con las filas repartidas la clase habla, porque cada grupo tiene que dictar la suya. El silencio aparece en el barrido, cuando se pide impugnar: nadie quiere contradecir a otro grupo en público. La maniobra de rescate es que el profesor impugne primero, y en concreto que impugne una celda que está bien, para que el grupo tenga que defenderla; defender lo propio es mucho más fácil que atacar lo ajeno y el efecto sobre el resto del aula es inmediato. La segunda maniobra es la votación sobre la celda dudosa, quién cree que esto es óptimo y quién no, con recuento escrito; si el reparto sale dividido, ya no hay que pedir nada. La tercera, para grupos que se han quedado en blanco al rellenar, es cambiar la pregunta abstracta por un caso: en lugar de si sirve con un mapa que cambia, se pregunta qué pasa si un operario deja un palé en medio del pasillo cuando el robot lleva media ruta hecha; con ese caso las cinco filas se rellenan solas, y de paso se prepara la replanificación de D^* con su cifra del doce por ciento de vértices reexpandidos (Corke, 2023, p. 184).

Cierre

La idea que debe quedar es que las cinco propiedades son cinco preguntas de proyecto, y que la elección se decide por la restricción más dura y no por la propiedad más prestigiosa: si la meta es estable, la transformada de distancia se amortiza; si el mapa cambia, hay que poder replanificar; si el espacio tiene seis dimensiones, hay que muestrear; y si hay obstáculos imprevistos, hace falta una capa reactiva por debajo. Conviene rematar con la frase del cuaderno, que es la que ordena la sesión: planificar empieza por preparar el mapa, no por elegir el algoritmo. Y con el anuncio que da continuidad a la tabla: mañana la primera pregunta de la clase es exactamente cuál elegiríais para un almacén con carretillas en movimiento, así que la tabla no se borra, se fotografía. Lo que no conviene hacer es declarar un ganador ni sugerir que A^* es el planificador que se usa siempre: es el más usado en robótica móvil por buenas razones y no vale para el brazo, y anticipar eso ahora es lo que hace que S37 tenga sentido. Tampoco conviene abrir la discusión de RRT^* y los planificadores asintóticamente óptimos más allá de la frase del guion, aunque alguien la pida: se remite a las referencias originales que el propio libro señala (Corke, 2023, p. 202) y al seminario.

La pregunta del costmap local en S36 (28–32 min): por qué no puede vivir en map

Objetivo

El guion lo formula como una pregunta directa a la clase dentro del tramo de costmaps, y su valor es desproporcionado respecto a lo que ocupa: es el momento en que la convención de frames de S34 se revela como una decisión de arquitectura de control y no como una regla administrativa. El apartado 36.2 lo dice ya, la estimación de localización corrige a saltos y un controlador reactivo que viviera en map vería teletransportarse los obstáculos con cada corrección, mientras que en odom el movimiento es continuo aunque derive, pero dicho por el profesor es una frase más y deducido por la clase es una idea que no se olvida. El objetivo, por tanto, es que la clase reconstruya la consecuencia a partir de lo que ya sabe: que map es exacta y discontinua, que odom es continua e inexacta, y que un lazo reactivo tolera error pero no discontinuidad. Hay un objetivo secundario que se cobra en el proyecto: quien entiende esto no vuelve a mezclar frames en un nodo propio, que es el error de integración más común y el más difícil de diagnosticar, porque el sistema funciona casi siempre.

Formato y tiempos

El tramo de 20 a 35 minutos es el de costmaps por capas, y la pregunta va al final, en los cuatro minutos que van del 28 al 32, dejando el resto para cerrar la comparación global frente a local. El reparto: en el minuto 28 se dibuja en la pizarra la ventana rodante del costmap local sobre el mapa global, se recuerda en una frase quién publica cada arista, la localización publica map a odom, la odometría publica odom a base_link, y se enuncia la pregunta; diez segundos de silencio. De 29 a 31, la discusión. De 31 a 32, la síntesis y la vuelta al hilo de la sesión. Un aviso de conducción: conviene tener a mano el árbol de transformadas del taller de S34 para proyectarlo si hace falta, porque la mitad de las respuestas fallidas vienen de haber olvidado quién publica qué, y volver a la figura resuelve eso en cinco segundos sin necesidad de explicar nada.

Pregunta de apertura

Literalmente, con el dibujo hecho: «Nav2 mantiene dos costmaps. El global está anclado a map y el local a odom. La pregunta es por qué no los dos a map, que es el frame exacto: ¿qué le pasaría al controlador local si sus obstáculos estuvieran expresados en map?» Y si en veinte segundos no hay respuesta, la pista mínima y no más: «Recordad qué le pasa a la arista map a odom cada vez que la localización se corrige.» La formulación funciona porque plantea la opción aparentemente mejor, usar el frame exacto para todo, y pide encontrarle el defecto; es mucho más productiva que preguntar por qué el local está en odom, que invita a repetir la convención.

Respuestas y resultados que van a salir, y qué hacer con cada uno

«**Porque map salta cuando la localización corrige**». Es la respuesta correcta y suele llegar de quien estuvo atento en S34; el trabajo consiste en no aceptarla como palabra y exigir la consecuencia física. La pregunta de vuelta es qué ve el controlador cuando eso pasa, y la respuesta que hay que arrancar es que ve el obstáculo desplazarse de golpe unos centímetros o unas decenas de centímetros sin que nada se haya movido, y que un controlador reactivo responde a eso con un volantazo. Conviene ponerle número usando el propio taller: con la odometría del ejercicio del cuaderno de S34, la corrección periódica producía una sierra cuyos dientes son exactamente el salto que vería el controlador. Ahí la clase entiende que el problema no es teórico.

«**Pero entonces el costmap local tiene los obstáculos mal colocados**». Es la objeción inteligente y la que hace valiosa la discusión; hay que reconocerla como correcta antes de resolverla. Sí, en odom la posición absoluta de los obstáculos deriva, y no importa por dos razones que conviene separar. La primera es que el costmap local es una ventana rodante centrada en el robot y de pocos metros, de

modo que lo que necesita es la posición relativa al robot, y ésa la dan los sensores directamente, sin pasar por la localización. La segunda es que la deriva en unos pocos segundos, que es la vida útil de una celda del costmap local, es despreciable. La formulación que resume las dos y conviene dictar: el controlador no necesita saber dónde está en el mundo, necesita saber qué tiene delante.

«**¿Y el planificador global no sufre lo mismo?**». Es la pregunta que cierra el razonamiento y hay que provocarla si no sale. El planificador global sí vive en map y sí ve los saltos, y no pasa nada por dos motivos: replanifica a baja frecuencia, de modo que un salto se absorbe en el siguiente plan, y su producto es un camino completo, no una consigna instantánea de velocidad. Conviene explicitar la simetría, porque es la idea general que se lleva: la frecuencia a la que trabaja un módulo decide qué clase de error tolera, el lazo rápido tolera sesgo y no tolera discontinuidad, el lazo lento tolera discontinuidad y no tolera sesgo. Es la misma lección que en el bloque 5 separaba el prealimentado del realimentado y en el bloque 6 separaba la odometría de la corrección por hitos.

«**Entonces mi nodo, ¿en qué frame trabaja?**». Es la mejor pregunta posible porque es la del proyecto, y hay que contestarla con una regla operativa y no con una teoría: lo que se usa para reaccionar, en odom o en el frame del robot; lo que se usa para saber dónde ir, en map; y nunca se mezclan dos frames en una misma resta sin pasar por el árbol. Conviene añadir el síntoma que produce incumplirla, porque es el que verán: el sistema funciona bien durante minutos y de pronto da un giro absurdo justo después de una corrección de localización, y el diagnóstico es casi imposible si no se sospecha del frame. Decirlo aquí, con nombre y apellidos, ahorra una tarde entera en diciembre.

El error de razonamiento más frecuente y cómo desmontarlo

El error dominante es creer que un frame más exacto es siempre preferible, y por tanto que trabajar en map es la buena práctica y trabajar en odom un apaño. Es un error de criterio y se desmonta con el experimento del cuaderno de S34, que es exactamente el que hace falta y conviene reproyectar en el minuto 31: la misma trayectoria con el error medido en los dos frames, la rampa monótona y sin cota en odom y la sierra acotada en map. La lectura hay que dictarla porque no es evidente: map es exacta pero discontinua y odom es continua pero inexacta, y no existe un frame que sea las dos cosas, de ahí que la convención obligue a tener ambos. Y de ahí sale además el criterio de diseño que el propio cuaderno señala y que sirve para el proyecto: la altura de los dientes de sierra es lo que decide la frecuencia con la que hay que localizar, porque si el robot deriva más de lo que el controlador tolera entre dos correcciones, hay que corregir más a menudo o mejorar la odometría. El segundo error, más silencioso, es suponer que la deriva de la odometría se puede corregir sobre la marcha sumándole la corrección de la localización dentro del propio nodo; se desmonta señalando que eso es exactamente reintroducir la discontinuidad en el frame continuo, es decir, deshacer con la mano derecha lo que la arquitectura hacía con la izquierda.

Si la clase no habla

El silencio aquí suele ser de falta de anclaje: han visto los frames hace cinco días y no los tienen frescos. La maniobra de rescate es proyectar el frames.pdf del taller y preguntar por partes, señalando: quién publica esta arista, con qué frecuencia, y qué le pasa cuando la localización corrige. Tres preguntas de una palabra devuelven el contexto y la pregunta grande se contesta después sola. La segunda maniobra es de teatro y funciona muy bien con este contenido concreto: se pide a un estudiante que camine hacia una silla mirando al suelo mientras el profesor, cada cuatro pasos, le dice que en realidad está medio metro a la izquierda de donde cree. La pregunta, si tuvieras que esquivar la silla, ¿te fías de mi corrección o de lo que ves?, produce la respuesta correcta en tres segundos y sin ninguna álgebra. La tercera, si el grupo está frío, es la votación binaria: quién pondría el costmap local en map y quién en odom, recuento en la pizarra y una razón de cada lado.

Cierre

La idea que debe quedar, en una frase que conviene escribir: la elección de frame de cada módulo es una decisión de arquitectura de control, y se toma por la clase de error que ese módulo tolera, no por la exactitud del frame. El corolario que se lleva al proyecto es la regla de las tres líneas que ya se ha dictado, reaccionar en odom, decidir en map, no mezclar sin pasar por el árbol, y la consigna de diagnóstico: cuando el robot haga algo absurdo justo después de una corrección de localización, el sospechoso es el frame. Lo que no conviene hacer es presentar esto como un detalle de implementación de Nav2, porque no lo es: es la razón de ser de la convención de frames y aparece igual en cualquier stack de navegación. Tampoco conviene abrir aquí la fusión de odometría con inercial ni el filtro que la produce, aunque alguien lo pregunte: se anota, se recuerda que el estimador es el del bloque 6 y se sigue, porque el tramo de árboles de comportamiento empieza en el minuto treinta y cinco y es el que aporta la novedad de la sesión.

La composición del stack completo y la puesta en común de riesgos en S37 (80–105 min)

Objetivo

Es la puesta en común más ambiciosa del bloque y la única cuyo producto se usa fuera de la clase: el diagrama del stack completo es el andamiaje con el que los equipos van a organizar el proyecto integrador y la defensa de S43. El guion la divide en tres movimientos, componer el diagrama entre todos, situar cada proyecto sobre él en equipos, y una puesta en común relámpago de un riesgo técnico por equipo, y los tres tienen objetivos distintos que conviene no confundir. Componer el diagrama es una tarea de síntesis: se trata de que la clase reconstruya, sin apuntes, la cadena que va de los sensores a las ruedas y de la cámara al brazo, y descubra al hacerlo que todas las piezas del semestre tienen sitio. Situar el proyecto es una tarea de proyección: obliga a cada equipo a declarar qué módulos usa, qué interfaces expone y qué parámetros va a tener que justificar. Y la puesta en común de riesgos es la más valiosa de las tres y la que suele salir mal por falta de tiempo: su objetivo es que cada equipo oiga los riesgos de los otros cuatro, porque el fallo que hunde un proyecto en diciembre casi nunca es el que ese equipo había previsto. La diferencia con la teoría es que el apartado 37.4 puede dibujar el diagrama y no puede colocar en él el proyecto de nadie.

Formato y tiempos

El guion asigna de 80 a 105 minutos, con la síntesis del bloque de 105 a 110 y el cuestionario de 110 a 120. El reparto que funciona: de 80 a 88, ocho minutos para componer el diagrama en la pizarra entre todos, con el profesor dibujando y la clase dictando. De 88 a 100, doce minutos de trabajo por equipos de proyecto sobre su propio caso, con el profesor rotando por las mesas, es tiempo real de tutoría y hay que protegerlo. De 100 a 105, cinco minutos de puesta en común de riesgos, un riesgo y una mitigación por equipo, sesenta segundos cada uno con reloj. Tres avisos de gestión. El primero: el diagrama lo dibuja el profesor y lo dicta la clase, nunca al contrario, porque si el profesor lo dicta se convierte en la última transparencia del bloque y se copia sin pensar. El segundo: los doce minutos de equipos se van en nada, así que hay que dar la consigna por escrito en la pizarra, qué bloques usa, qué interfaz expone cada módulo propio, qué plugins y parámetros hay que justificar, un riesgo, y rotar con reloj, dos minutos por equipo. El tercero: la puesta en común de riesgos tiene que empezar a las cien en punto, aunque los equipos no hayan acabado, porque es la parte que no se puede recuperar y el cuestionario no se puede mover.

Preguntas de apertura

Para el diagrama, literalmente: «Vamos a dibujar el robot completo del curso, y lo vais a dictar vosotros. Empiezo por la izquierda con una caja que dice sensores. ¿Qué va después, y qué viaja por la flecha?» Insistir en que digan lo que viaja por la flecha, y no sólo el nombre de la caja siguiente, es lo que convierte el ejercicio en síntesis: las flechas son interfaces con tipo y con QoS, y ése es el aprendizaje del bloque. Para los riesgos, en el minuto 100 y con el reloj a la vista: «Un riesgo técnico y una mitigación, en sesenta segundos, y no vale decir que os falta tiempo. Quiero un riesgo que se pueda diagnosticar: qué veríais si ocurriera.»

Respuestas y resultados que van a salir, y qué hacer con cada uno

El diagrama se atasca siempre en el mismo punto: entre el mapa y el planificador. La clase dicta sin problema sensores, percepción, mapa y ruedas, y se queda muda cuando hay que decir quién convierte una meta en velocidades. Es el momento de rescatar el reparto de servidores de S36, planificador global sobre el costmap global, controlador local produciendo /cmd_vel sobre el costmap local, comportamientos de recuperación, y el orquestador por encima recibiendo la acción de navegación (docs.nav2.org, Navigation Concepts), y de pedir que digan dónde encaja la localización. Que ese trozo cueste es buena señal y hay que decirlo: es el único trozo del diagrama que no se ve desde fuera del sistema.

Nadie dibuja TF2 espontáneamente. Ocurre todos los años y es el hallazgo pedagógico del ejercicio. La clase dibuja las cajas y las flechas de datos y se olvida de la columna vertebral geométrica, porque TF2 no está en el camino de ningún dato concreto: está debajo de todos. Cuando el diagrama esté acabado hay que preguntar qué falta, dejar que no lo encuentren, y entonces dibujar TF2 como una banda que atraviesa el dibujo entero. La frase que conviene decir ahí es que casi todas las cajas del diagrama piden transformadas y casi ninguna las publica, y que por eso un árbol roto no da error en ningún nodo y estropea el sistema completo.

«Nuestro riesgo es que no nos dé tiempo». Es lo que dirá el primer equipo y hay que cortarlo con amabilidad y sin excepciones, porque si se acepta lo repiten los cuatro siguientes y los cinco minutos se pierden. La reformulación que hay que exigir es técnica y diagnosticable: qué módulo concreto es el que puede no llegar, qué síntoma tendría y qué se hace en su lugar. Conviene tener preparados los tres riesgos que el propio guion nombra, para ofrecerlos como plantilla si un equipo no encuentra el suyo: QoS incompatibles entre el driver y el stack de navegación, TF sin sincronizar en la cámara, y el planificador del brazo que no encuentra camino con la pinza cargada.

«El nuestro es que el brazo no encuentre camino». Es un riesgo bien formulado y hay que aprovecharlo para cobrar la teoría de la sesión: la mitigación no es cambiar de planificador, es revisar la escena de planificación con la geometría real de la herramienta y de la pieza (moveit.picknik.ai, Concepts: Planning Scene), porque la causa habitual es que la pinza cargada colisiona con algo que el modelo no incluía, y el planificador está haciendo su trabajo. Y hay una segunda mitigación que conviene nombrar porque es la que nadie propone: aceptar el fallo y diseñar qué hace el sistema cuando no hay camino, que es exactamente el papel de las ramas de recuperación de un árbol de comportamiento (docs.nav2.org, Detailed Behavior Tree Walkthrough). Un proyecto que sólo funciona cuando todo va bien no está terminado.

El error de razonamiento más frecuente y cómo desmontarlo

El error dominante en esta puesta en común es de arquitectura y aparece en cuanto los equipos sitúan su proyecto: confundir el diagrama de módulos con un diagrama de bloques secuencial, en el que cada caja termina su trabajo y entrega el resultado a la siguiente. De esa lectura salen las dos decisiones que arruinan integraciones: implementar como topic con banderas lo que semánticamente es una acción,

perdiendo cancelación y resultado, que es el error de diseño más común en los proyectos de años anteriores según el apartado 33.1, y suponer que todo el mundo trabaja a la misma frecuencia y con el mismo sello de tiempo. Se desmonta con el propio diagrama, pidiendo a la clase que anote la frecuencia junto a cada flecha: el láser a cuarenta hercios, el controlador local a diez o veinte, el planificador global cada uno o dos segundos, el árbol de comportamiento a su ritmo, y el lazo de par del bloque 5 a kilohercios. Con las frecuencias escritas, el dibujo deja de parecer una cadena de montaje y se ve lo que es, un conjunto de lazos concurrentes que comparten datos con sellos de tiempo distintos; y de ahí sale sola la razón de existir de TF2 con su búfer temporal (docs.ros.org, tutorial Introducing tf2). El segundo error, muy frecuente en la parte de equipos, es declarar que se usará un módulo sin declarar la interfaz por la que se conecta; se desmonta preguntando el nombre y el tipo del topic, porque un módulo sin interfaz declarada es un módulo que no se ha diseñado.

Si la clase no habla

En la composición del diagrama el silencio es raro pero cuando aparece es fatal, porque el profesor se queda solo delante de una pizarra vacía. La maniobra de rescate es empezar por el final y no por el principio: se dibuja la caja de los motores a la derecha y se pregunta qué llega hasta ahí y quién lo manda. Ir hacia atrás desde el actuador es mucho más fácil que ir hacia delante desde el sensor, porque cada caja tiene una única respuesta razonable. La segunda maniobra es repartir el dibujo por bloques del curso: se pide al fondo del aula la parte del bloque 6, al centro la del 7 y a la primera fila la del 4 y el 5; con la responsabilidad repartida siempre hay alguien que habla. En la puesta en común de riesgos el problema es el contrario, que hablan de más, y la única herramienta es el reloj visible y el corte a los sesenta segundos, anunciado antes de empezar y aplicado sin excepciones desde el primer equipo, porque la equidad de estos cinco minutos es el tiempo.

Cierre

La idea que debe quedar es la que el apartado 37.4 formula y que aquí se ha construido en la pizarra: cada flecha del diagrama es una interfaz de ROS 2 con su tipo y su calidad de servicio, y cada caja es un conjunto de nodos con parámetros que hay que justificar, no sólo copiar. Conviene añadir el corolario que la puesta en común de riesgos ha demostrado: los proyectos no fallan por el algoritmo, fallan por las flechas, QoS, frames, sellos de tiempo y semántica de interfaz. Y el encargo operativo: el diagrama se fotografía, cada equipo lo incorpora a su memoria con su propio recorrido marcado, y los cinco riesgos se publican en el campus para todos, porque el riesgo de un equipo es el aviso de otro. Lo que no conviene hacer es cerrar el bloque prometiendo que si el diagrama está claro el proyecto saldrá bien: no es cierto y baja la guardia justo cuando hay que subirla. Tampoco conviene rematar con una lista de recomendaciones de última hora sobre versiones y paquetes: se publica por escrito en el campus, que es donde se puede consultar, y los cinco minutos que quedan se usan para la síntesis del bloque, que es lo que el cuestionario va a preguntar.

Bibliografía citada en este bloque

- **Corke, P.** Robotics, Vision and Control: Fundamental Algorithms in Python. 3.^a ed., Springer, 2023. Cap. 5 (Navigation, pp. 161-214): representaciones de mapa (p. 166), búsqueda en grafos BFS/UCS (pp. 168-173), A* y admisibilidad (pp. 174-175), mapas de ocupación (p. 177), transformada de distancia (pp. 177-181), inflado de obstáculos (p. 181), D* y mapas de coste (pp. 182-184), fases de planificación y consulta (p. 184), PRM (pp. 185-187), RRT con primitivas de Dubins (pp. 196-199), A* frente a Dijkstra (p. 199), referencias de LaValle y Karaman et al. (p. 202).
- **Lynch, K. M. y Park, F. C.** Modern Robotics: Mechanics, Planning, and Control. Cambridge University Press, 2017. Cap. 10 (Motion Planning, pp. 353-398): formulación y C_free (p. 353), planificación de caminos y problema del piano (p. 355), completitud y propiedades (pp. 355-356), C-obstáculos (pp. 358-361), búsqueda A* (pp. 365-367), métodos de muestreo (p. 378), algoritmo RRT (p. 379), planificadores locales y RRT bidireccional (p. 382), RRT* (p. 383), PRM (pp. 384-385), campos potenciales y mínimos locales (pp. 386-388).
- **Thrun, S., Burgard, W. y Fox, D.** Probabilistic Robotics. MIT Press, 2005. Cap. 9 (Occupancy Grid Mapping, pp. 281-307): mapa como campo de variables aleatorias binarias y posterior (p. 284), utilidad como posprocesado tras SLAM (p. 284), algoritmo occupancy_grid_mapping con log-odds (p. 286). Cap. 8: localización de Monte Carlo, fundamento de AMCL (citado como capítulo).
- **Documentación oficial de ROS 2:** docs.ros.org, versión LTS Jazzy Jalisco. Citada por nombre de documento y sección, sin página: conceptos (The ROS 2 graph; About Quality of Service settings; About different ROS 2 DDS/RMW vendors; About tf2) y tutoriales (Understanding nodes / topics / services / actions / parameters; Creating a launch file; Creating a workspace; Creating a package; Using colcon to build packages; Writing a simple publisher and subscriber (Python); Introducing tf2).
- **Documentación oficial de Gazebo:** gazebosim.org/docs (simulador, SDF, sensores) y documentación del paquete ros_gz (puente ros_gz_bridge). Citadas por nombre y sección, sin página.
- **Documentación oficial de Nav2:** docs.nav2.org. Citada por sección, sin página: Navigation Concepts (servidores, lifecycle nodes), Configuration Guide (Costmap 2D y capas static/obstacle/inflation; AMCL; planner/controller/behavior/smoothers servers), Behavior Trees y Detailed Behavior Tree Walkthrough.
- **Documentación oficial de MoveIt 2:** moveit.picknik.ai. Citada por sección, sin página: Concepts (Move Group; Planning Scene; Motion Planning; OMPL Planner), MoveIt Setup Assistant (SRDF) y tutoriales de la MoveGroupInterface (computeCartesianPath).
- **Convenciones ROS:** REP 105, Coordinate Frames for Mobile Platforms (convención map → odom → base_link), citada por designación.